DeepL
Subscribe to DeepL Pro to translate larger documents.
Visit www.DeepL.com/pro for more information.

CHƯƠNG 1

Bức tranh tổng quan về Học máy

Cách đây không lâu, nếu bạn cầm điện thoại lên và yêu cầu nó chỉ đường về nhà, nó sẽ phớt lờ bạn — và mọi người sẽ nghi ngờ bạn có bị mất trí không. Nhưng học máy không còn là khoa học viễn tưởng nữa: hàng tỷ người sử dụng nó hàng ngày. Và sự thật là nó đã tồn tại từ nhiều thập kỷ nay trong một số ứng dụng chuyên biệt, chẳng hạn như nhận dạng ký tự quang học (OCR). Ứng dụng học máy đầu tiên thực sự trở nên phổ biến, cải thiện cuộc sống của hàng trăm triệu người, đã âm thầm chiếm lĩnh thế giới từ những năm 1990: *bộ lọc thư rác*. Nó không hẳn là một robot có ý thức, nhưng về mặt kỹ thuật, nó vẫn được coi là học máy: nó đã học tốt đến mức bạn hiếm khi phải đánh dấu một email là thư rác nữa. Sau đó, nhờ vào dữ liệu lớn, sự cải tiến về phần cứng và một số đột phá về thuật toán, hàng trăm ứng dụng ML đã ra đời và hiện đang âm thầm vận hành hàng trăm sản phẩm và tính năng mà bạn sử dụng hàng ngày: lời nhắc bằng giọng nói, dịch tự động, tìm kiếm hình ảnh, đề xuất sản phẩm và nhiều hơn nữa. Cuối cùng là sự xuất hiện của ChatGPT, Gemini (trước đây là Bard), Claude, Perplexity và nhiều chatbot khác: AI không còn chỉ là công cụ vận hành các dịch vụ ở hậu trường, mà chính nó *đã trở thành* dịch vụ đó.

Học máy bắt đầu từ đâu và kết thúc ở đâu? Cụ thể thì việc một cỗ máy *"học"* được điều gì có nghĩa là gì? Nếu tôi tải xuống bản sao của tất cả các bài viết trên Wikipedia, liệu máy tính của tôi có thực sự học được điều gì không? Liệu nó có đột nhiên trở nên thông minh hơn không? Trong chương này, tôi sẽ bắt đầu bằng việc làm rõ học máy là gì và tại sao bạn có thể muốn sử dụng nó.

Sau đó, trước khi chúng ta bắt đầu khám phá "lục địa" học máy, chúng ta sẽ xem qua bản đồ và tìm hiểu về các khu vực chính cùng những điểm mốc nổi bật: học có giám sát so với học không giám sát và các biến thể của chúng, học trực tuyến so với học theo lô, học dựa trên mẫu so với học dựa trên mô hình. Tiếp theo, chúng ta sẽ xem xét quy trình làm việc của một dự án học máy điển hình, thảo luận về những thách thức chính mà bạn có thể gặp phải, và tìm hiểu cách đánh giá và tinh chỉnh hệ thống học máy.

Chương này giới thiệu rất nhiều khái niệm cơ bản (và thuật ngữ chuyên ngành) mà mọi nhà khoa học dữ liệu đều cần nắm vững. Đây sẽ là một cái nhìn tổng quan ở mức độ cao (đây là chương duy nhất không có nhiều mã nguồn), tất cả đều khá đơn giản, nhưng mục tiêu của tôi là đảm bảo mọi thứ đều rõ ràng như pha lê đối với bạn trước khi chúng ta tiếp tục phần còn lại của cuốn sách. Vậy hãy pha một tách cà phê và bắt đầu thôi!



Nếu bạn đã quen thuộc với những kiến thức cơ bản về học máy, bạn có thể bỏ qua trực tiếp đến Chương 2. Nếu bạn chưa chắc chắn, hãy thử trả lời tất cả các câu hỏi được liệt kê ở cuối chương trước khi tiếp tục.

Học máy là gì?

Học máy là khoa học (và nghệ thuật) lập trình máy tính để chúng có thể *học từ dữ liệu*.

Dưới đây là một định nghĩa tổng quát hơn một chút:

> [Học máy là] lĩnh vực nghiên cứu giúp máy tính có khả năng học hỏi mà không cần được lập trình rõ ràng.
>
> —Arthur Samuel, 1959

Và một định nghĩa mang tính kỹ thuật hơn:

> Một chương trình máy tính được coi là học hỏi từ kinh nghiệm *E* liên quan đến một nhiệm vụ *T* và một thước đo hiệu suất *P*, nếu hiệu suất của nó trên *T*, được đo bằng *P*, được cải thiện nhờ kinh nghiệm *E*.
>
> —Tom Mitchell, 1997

Bộ lọc thư rác của bạn là một chương trình học máy mà khi được cung cấp các ví dụ về email rác (được người dùng gắn cờ) và các ví dụ về email thông thường (không phải thư rác, còn gọi là "ham"), có thể học cách gắn cờ thư rác. Các ví dụ mà hệ thống sử dụng để học được gọi là *tập huấn luyện*. Mỗi ví dụ huấn luyện được gọi là một *trường hợp huấn luyện* (hoặc *mẫu*). Phần của hệ thống học máy chịu trách nhiệm học và đưa ra dự đoán được gọi là *mô hình*. Mạng nơ-ron và rừng ngẫu nhiên là những ví dụ về mô hình.

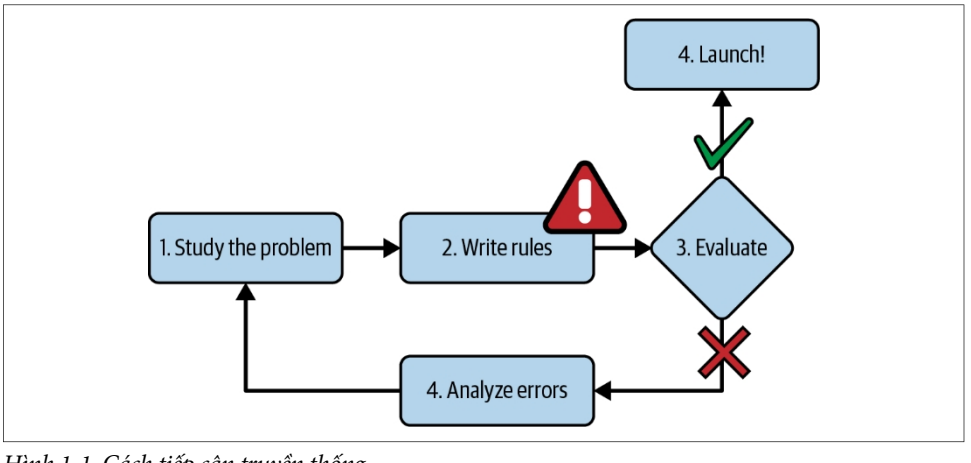
Trong trường hợp này, tác vụ *T* là đánh dấu thư rác cho các email mới, kinh nghiệm *E* là *dữ liệu huấn luyện*, và thước đo hiệu suất *P* cần được định nghĩa; ví dụ, bạn có thể sử dụng tỷ lệ email được phân loại chính xác. Thước đo hiệu suất cụ thể này được gọi là *độ chính xác*, và nó thường được sử dụng trong các tác vụ phân loại (chúng ta sẽ thảo luận về một số thước đo khác trong Chương 3).

Nếu bạn chỉ tải xuống một bản sao của tất cả các bài viết trên Wikipedia, máy tính của bạn sẽ có nhiều dữ liệu hơn, nhưng nó sẽ không đột nhiên trở nên giỏi hơn trong bất kỳ nhiệm vụ nào. Đây không phải là học máy.

Tại sao nên sử dụng học máy?

Hãy xem xét cách bạn sẽ viết một bộ lọc thư rác bằng các kỹ thuật lập trình truyền thống (Hình 1-1):

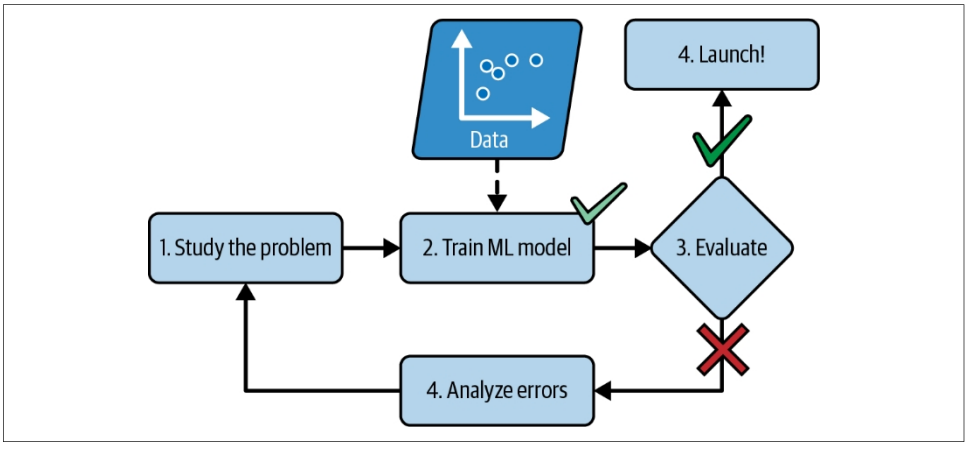
1. Đầu tiên, bạn sẽ phân tích xem thư rác thường trông như thế nào. Bạn có thể nhận thấy rằng một số từ hoặc cụm từ (chẳng hạn như "4U", "thẻ tín dụng", "miễn phí" và "tuyệt vời") thường xuất hiện nhiều trong dòng tiêu đề. Có lẽ bạn cũng sẽ nhận thấy một vài mẫu khác trong tên người gửi, nội dung email và các phần khác của email.
2. Bạn sẽ viết một thuật toán phát hiện cho từng mẫu mà bạn nhận thấy, và chương trình của bạn sẽ đánh dấu các email là spam nếu phát hiện được một số mẫu này.
3. Bạn sẽ kiểm tra chương trình của mình và lặp lại các bước 1 và 2 cho đến khi chương trình đủ tốt để đưa vào sử dụng.



Hình 1-1. Cách tiếp cận truyền thống

Vì vấn đề này khá phức tạp, chương trình của bạn có thể sẽ trở thành một danh sách dài các quy tắc phức tạp — khá khó để duy trì.

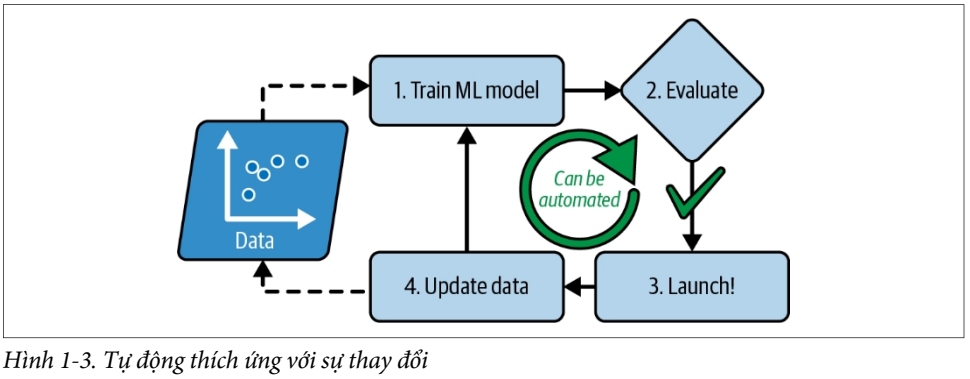
Ngược lại, bộ lọc thư rác dựa trên các kỹ thuật học máy sẽ tự động học được những từ và cụm từ nào là dấu hiệu dự báo thư rác hiệu quả bằng cách phát hiện các mẫu từ xuất hiện với tần suất bất thường trong các ví dụ thư rác so với các ví dụ thư hợp lệ (Hình 1-2). Chương trình này ngắn gọn hơn, dễ bảo trì hơn và rất có thể sẽ chính xác hơn.



Hình 1-2. Phương pháp học máy

Nếu những kẻ gửi thư rác nhận thấy rằng tất cả các email của họ có chứa từ "4U" đều bị chặn thì sao? Họ có thể bắt đầu viết "For U" thay thế. Bộ lọc thư rác sử dụng các kỹ thuật lập trình truyền thống sẽ cần được cập nhật để đánh dấu các email có chứa từ "For U". Nếu những kẻ gửi thư rác tiếp tục tìm cách vượt qua bộ lọc thư rác của bạn, bạn sẽ phải tiếp tục viết các quy tắc mới mãi mãi.

Ngược lại, bộ lọc thư rác dựa trên các kỹ thuật học máy sẽ tự động nhận ra rằng cụm từ "For U" xuất hiện với tần suất bất thường trong các thư rác đã được người dùng gắn cờ, và bắt đầu tự động gắn cờ chúng mà không cần sự can thiệp của bạn (Hình 1-3).

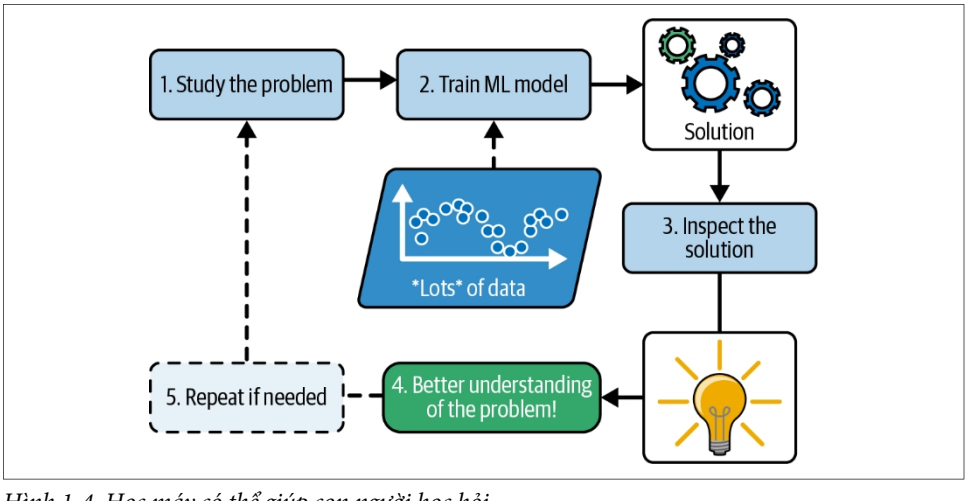


Hình 1-3. Tự động thích ứng với sự thay đổi

Một lĩnh vực khác mà học máy tỏa sáng là các vấn đề quá phức tạp đối với các phương pháp truyền thống hoặc chưa có thuật toán nào được biết đến. Ví dụ, hãy xem xét nhận dạng giọng nói. Giả sử bạn muốn bắt đầu từ những điều đơn giản và viết một chương trình có khả năng phân biệt các từ "one" và "two". Bạn có thể nhận thấy rằng từ "two" bắt đầu bằng một âm cao ("T"), do đó bạn có thể lập trình cứng một thuật toán đo cường độ âm cao và sử dụng điều đó để phân biệt "one" và "two"—nhưng rõ ràng kỹ thuật này sẽ không thể mở rộng để xử lý hàng nghìn từ được phát âm bởi hàng triệu

người khác nhau trong môi trường ồn ào và bằng hàng chục ngôn ngữ. Giải pháp tốt nhất (ít nhất là hiện nay) là viết một thuật toán có khả năng tự học, dựa trên nhiều bản ghi âm mẫu cho mỗi từ.

Cuối cùng, học máy có thể giúp con người học hỏi (Hình 1-4). Các mô hình học máy có thể được kiểm tra để xem chúng đã học được gì (mặc dù đối với một số mô hình, việc này có thể khá phức tạp). Ví dụ, sau khi bộ lọc thư rác đã được huấn luyện trên đủ lượng thư rác, người ta có thể dễ dàng kiểm tra nó để tìm ra danh sách các từ và tổ hợp từ mà nó cho là các yếu tố dự báo tốt nhất cho thư rác. Đôi khi điều này sẽ tiết lộ những mối tương quan không ngờ tới hoặc các xu hướng mới, từ đó giúp hiểu rõ hơn về vấn đề. Việc đào sâu vào lượng dữ liệu khổng lồ để khám phá các mẫu ẩn được gọi là *khai thác dữ liệu*, và học máy rất xuất sắc trong lĩnh vực này.



Hình 1-4. Học máy có thể giúp con người học hỏi

Tóm lại, học máy rất phù hợp cho:

- Các vấn đề mà các giải pháp hiện có đòi hỏi nhiều công sức và bảo trì, chẳng hạn như danh sách quy tắc dài (mô hình học máy thường có thể đơn giản hóa mã và hoạt động hiệu quả hơn so với phương pháp truyền thống)
- Các vấn đề phức tạp mà việc sử dụng phương pháp truyền thống không mang lại giải pháp tốt (các kỹ thuật học máy tốt nhất có thể tìm ra giải pháp)
- Môi trường biến động (hệ thống học máy có thể dễ dàng được huấn luyện lại trên dữ liệu mới, luôn giữ cho hệ thống cập nhật)
- Thu thập thông tin chi tiết về các vấn đề phức tạp và lượng dữ liệu lớn

Ví dụ về các ứng dụng

Hãy xem xét một số ví dụ cụ thể về các tác vụ học máy, cùng với các kỹ thuật có thể giải quyết chúng:

Phân tích hình ảnh các sản phẩm trên dây chuyền sản xuất để tự động phân loại chúng
: Đây là phân loại hình ảnh, thường được thực hiện bằng cách sử dụng mạng nơ-ron xoắn (CNN; xem Chương 12) hoặc bộ chuyển đổi thị giác (xem Chương 16).

Phát hiện khối u trong ảnh chụp não
: Đây là phân đoạn hình ảnh ngữ nghĩa, trong đó mỗi pixel trong hình ảnh được phân loại (vì chúng ta muốn xác định vị trí và hình dạng chính xác của khối u), thường sử dụng CNN hoặc bộ chuyển đổi thị giác.

Phân loại tự động các bài báo
: Đây là xử lý ngôn ngữ tự nhiên (NLP), cụ thể hơn là phân loại văn bản, có thể được giải quyết bằng cách sử dụng mạng nơ-ron tái diễn (RNN) và CNN, nhưng các bộ chuyển đổi hoạt động hiệu quả hơn (xem Chương 15).

Tự động đánh dấu các bình luận xúc phạm trên các diễn đàn thảo luận
: Đây cũng là phân loại văn bản, sử dụng các công cụ NLP tương tự.

Tóm tắt tự động các tài liệu dài
: Đây là một nhánh của NLP gọi là tóm tắt văn bản, cũng sử dụng các công cụ tương tự.

Ước tính nguy cơ di truyền của một người đối với một bệnh cụ thể bằng cách phân tích chuỗi DNA rất dài
: Nhiệm vụ này đòi hỏi phải phát hiện các mẫu phân bố rải rác trên các chuỗi rất dài, và đây chính là lĩnh vực mà các mô hình không gian trạng thái (SSMs) đặc biệt phát huy hiệu quả (xem "Mô hình không gian trạng thái (SSMs)" tại *https://homl.info*).

Tạo chatbot hoặc trợ lý cá nhân
: Việc này liên quan đến nhiều thành phần NLP, bao gồm hiểu ngôn ngữ tự nhiên (NLU) và các mô-đun trả lời câu hỏi.

Dự báo doanh thu của công ty bạn trong năm tới, dựa trên nhiều chỉ số hiệu suất
: Đây là một tác vụ hồi quy (tức là dự đoán các giá trị) có thể được giải quyết bằng bất kỳ mô hình hồi quy nào, chẳng hạn như mô hình hồi quy tuyến tính hoặc hồi quy đa thức (xem Chương 4), máy vectơ hỗ trợ hồi quy (xem phụ lục trực tuyến về SVM tại *https://homl.info*), rừng ngẫu nhiên hồi quy (xem Chương 6) hoặc mạng nơ-ron nhân tạo (xem Chương 9). Nếu bạn muốn tính đến chuỗi các chỉ số hiệu suất trong quá khứ, bạn có thể sử dụng RNN, CNN hoặc transformer (xem Chương 13 đến 15).

Làm cho ứng dụng của bạn phản hồi với các lệnh thoại
: Đây là nhận dạng giọng nói, đòi hỏi phải xử lý các mẫu âm thanh. Vì đây là các chuỗi dài và phức tạp, chúng thường được xử lý bằng RNN, CNN hoặc transformer (xem Chương 13 đến 15).

Phát hiện gian lận thẻ tín dụng
: Đây là việc phát hiện sự bất thường, có thể được giải quyết bằng cách sử dụng rừng cô lập, mô hình hỗn hợp Gaussian (xem Chương 8) hoặc bộ mã hóa tự động (xem Chương 18).

Phân đoạn khách hàng dựa trên các giao dịch mua hàng của họ để bạn có thể thiết kế chiến lược tiếp thị khác nhau cho từng phân khúc
: Đây là phân cụm, có thể thực hiện bằng cách sử dụng k-means, DBSCAN và các phương pháp khác (xem Chương 8).

Thể hiện một tập dữ liệu phức tạp, đa chiều dưới dạng sơ đồ rõ ràng và mang tính thông tin Đây là trực quan hóa dữ liệu, thường liên quan đến các kỹ thuật giảm chiều (xem Chương 7).

Đề xuất một sản phẩm mà khách hàng có thể quan tâm, dựa trên lịch sử mua hàng. Đây là một hệ thống đề xuất. Một cách tiếp cận là đưa lịch sử mua hàng (và các thông tin khác về khách hàng) vào một mạng nơ-ron nhân tạo (xem Chương 9), và khiến nó đưa ra dự đoán về sản phẩm mà khách hàng có khả năng mua tiếp theo cao nhất. Mạng nơ-ron này thường được huấn luyện dựa trên chuỗi lịch sử mua hàng của tất cả khách hàng.

Xây dựng một bot thông minh cho trò chơi
: Vấn đề này thường được giải quyết bằng cách sử dụng học tăng cường (RL; xem Chương 19), một nhánh của học máy nhằm huấn luyện các tác nhân (chẳng hạn như bot) chọn những hành động sẽ tối đa hóa phần thưởng của chúng theo thời gian (ví dụ: một bot có thể nhận được phần thưởng mỗi khi người chơi mất một số điểm sinh mạng), trong một môi trường nhất định (chẳng hạn như trò chơi). Chương trình AlphaGo nổi tiếng đã đánh bại nhà vô địch thế giới trong trò chơi cờ vây được xây dựng bằng cách sử dụng RL.

Danh sách này có thể còn dài, nhưng hy vọng nó sẽ giúp bạn cảm nhận được phạm vi và độ phức tạp đáng kinh ngạc của các nhiệm vụ mà học máy có thể giải quyết, cũng như các loại kỹ thuật mà bạn sẽ sử dụng cho từng nhiệm vụ.

Các loại hệ thống học máy

Có rất nhiều loại hệ thống học máy khác nhau, do đó sẽ rất hữu ích nếu phân loại chúng thành các danh mục rộng, dựa trên các tiêu chí sau:

- Cách chúng được hướng dẫn trong quá trình huấn luyện (có giám sát, không giám sát, bán giám sát, tự giám sát và các loại khác)
- Chúng có thể học dần dần trong quá trình thực hiện hay không (học trực tuyến so với học theo lô)

- Chúng hoạt động bằng cách đơn giản so sánh các điểm dữ liệu mới với các điểm dữ liệu đã biết, hay bằng cách phát hiện các mẫu trong dữ liệu huấn luyện và xây dựng mô hình dự đoán, tương tự như cách các nhà khoa học làm (học dựa trên thực thể so với học dựa trên mô hình)

Các tiêu chí này không loại trừ lẫn nhau; bạn có thể kết hợp chúng theo bất kỳ cách nào bạn muốn. Ví dụ, một bộ lọc thư rác tiên tiến có thể học ngay lập tức bằng cách sử dụng mô hình mạng nơ-ron sâu được huấn luyện dựa trên các ví dụ về thư rác và thư hợp lệ do con người cung cấp; điều này khiến nó trở thành một hệ thống học trực tuyến, dựa trên mô hình và có giám sát.

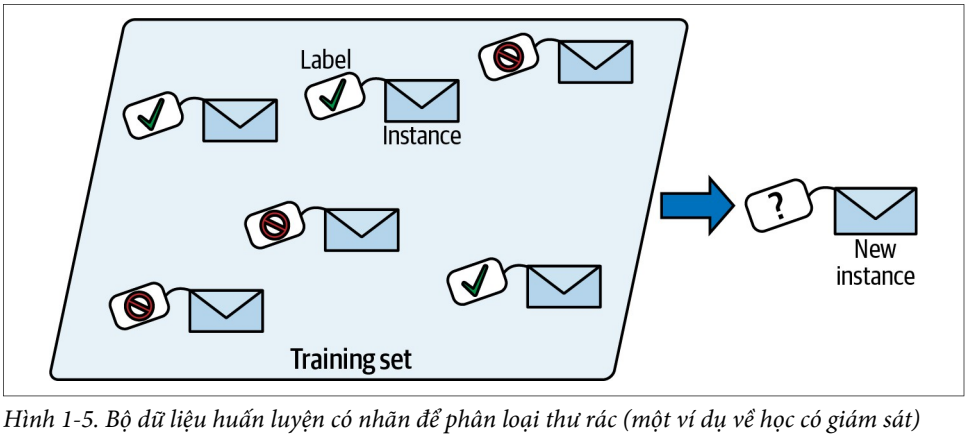
Hãy xem xét từng tiêu chí này kỹ hơn một chút.

Giám sát quá trình huấn luyện

Các hệ thống ML có thể được phân loại theo mức độ và loại giám sát mà chúng nhận được trong quá trình huấn luyện. Có nhiều loại, nhưng chúng ta sẽ thảo luận về những loại chính: học có giám sát, học không có giám sát, học tự giám sát, học bán giám sát và học củng cố.

Học có giám sát

Trong *học có giám sát*, tập dữ liệu huấn luyện mà bạn cung cấp cho thuật toán bao gồm các giải pháp mong muốn, được gọi là *nhãn* (Hình 1-5).



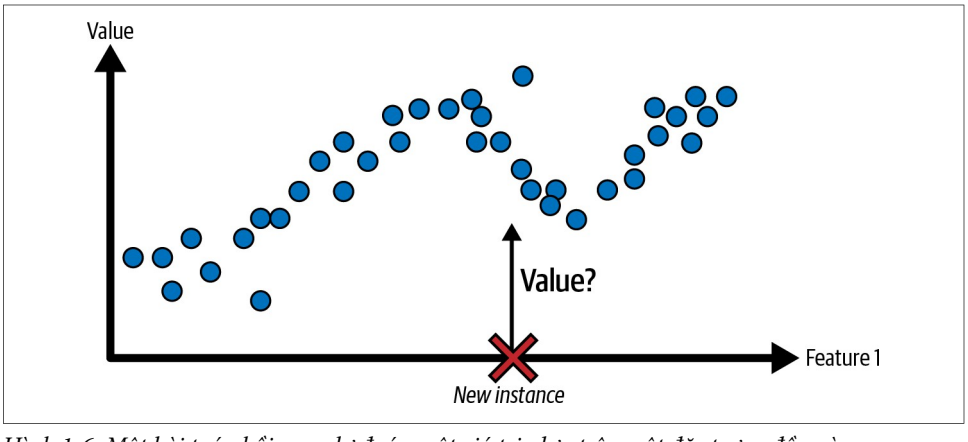
Hình 1-5. Bộ dữ liệu huấn luyện có nhãn để phân loại thư rác (một ví dụ về học có giám sát)

Một tác vụ học có giám sát điển hình là *phân loại*. Bộ lọc thư rác là một ví dụ điển hình: nó được huấn luyện bằng nhiều email mẫu cùng với *lớp* của chúng (thư rác hoặc thư hợp lệ), và nó phải học cách phân loại các email mới.

Một tác vụ điển hình khác là dự đoán một giá trị số *mục tiêu*, chẳng hạn như giá của một chiếc ô tô, dựa trên một tập hợp các *đặc điểm* (số dặm đã chạy, tuổi thọ, thương hiệu, v.v.). Loại tác vụ này được gọi là

hồi quy (Hình 1-6).[1] Để huấn luyện hệ thống, bạn cần cung cấp cho nó nhiều ví dụ về ô tô, bao gồm cả các đặc trưng và giá trị mục tiêu của chúng (tức là giá cả).

Lưu ý rằng một số mô hình hồi quy cũng có thể được sử dụng cho phân loại, và ngược lại. Ví dụ, *hồi quy logistic* thường được sử dụng cho phân loại, vì nó có thể đưa ra một giá trị tương ứng với xác suất thuộc về một lớp nhất định (ví dụ: 20% khả năng là thư rác).



Hình 1-6. Một bài toán hồi quy: dự đoán một giá trị, dựa trên một đặc trưng đầu vào (thường có nhiều đặc trưng đầu vào và đôi khi có nhiều giá trị đầu ra)



Các từ *"target"* và *"label"* thường được coi là từ đồng nghĩa trong học có giám sát, nhưng *"target"* phổ biến hơn trong các tác vụ hồi quy và *"label"* phổ biến hơn trong các tác vụ phân loại. Hơn nữa, *các đặc trưng* đôi khi được gọi là *biến dự báo* hoặc *thuộc tính*. Các thuật ngữ này có thể để cập đến các mẫu riêng lẻ (ví dụ: "đặc trưng số dặm của chiếc xe này bằng 15.000") hoặc tất cả các mẫu (ví dụ: "đặc trưng số dặm có mối tương quan mạnh với giá").

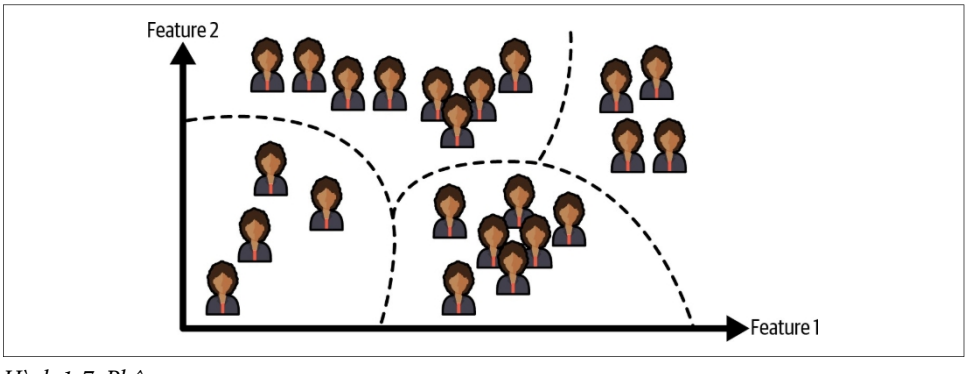
Học không có giám sát

Trong *học không có giám sát*, như bạn có thể đoán, dữ liệu huấn luyện không có nhãn. Hệ thống cố gắng học mà không cần người hướng dẫn.

Ví dụ, giả sử bạn có nhiều dữ liệu về người truy cập blog của mình. Bạn có thể muốn chạy một thuật toán *phân cụm* để cố gắng phát hiện các nhóm người truy cập có đặc điểm tương tự (Hình 1-7). Các đặc trưng có thể bao gồm nhóm tuổi của người dùng, khu vực họ sinh sống, sở thích của họ, thời gian

1 Thông tin thú vị: cái tên nghe có vẻ kỳ lạ này là một thuật ngữ thống kê do Francis Galton đưa ra khi ông nghiên cứu hiện tượng con cái của những người cao thường có chiều cao thấp hơn cha mẹ. Vì những đứa trẻ này thấp hơn, ông gọi hiện tượng này *là "hồi quy về trung bình"*. Tên gọi này sau đó được áp dụng cho các phương pháp mà ông sử dụng để phân tích mối tương quan giữa các biến.

thời gian truy cập, v.v. Bạn không cần phải chỉ định cho thuật toán nhóm nào mà người truy cập thuộc về: thuật toán sẽ tự động tìm ra các mối liên hệ đó mà không cần sự trợ giúp của bạn. Ví dụ, thuật toán có thể nhận thấy rằng 40% người truy cập của bạn là thanh thiếu niên yêu thích truyện tranh và thường đọc blog của bạn sau giờ học, trong khi 20% là người trưởng thành thích khoa học viễn tưởng và truy cập vào cuối tuần. Nếu bạn sử dụng thuật toán *phân cụm phân cấp*, thuật toán này cũng có thể chia nhỏ từng nhóm thành các nhóm nhỏ hơn. Điều này có thể giúp bạn định hướng các bài đăng cho từng nhóm.



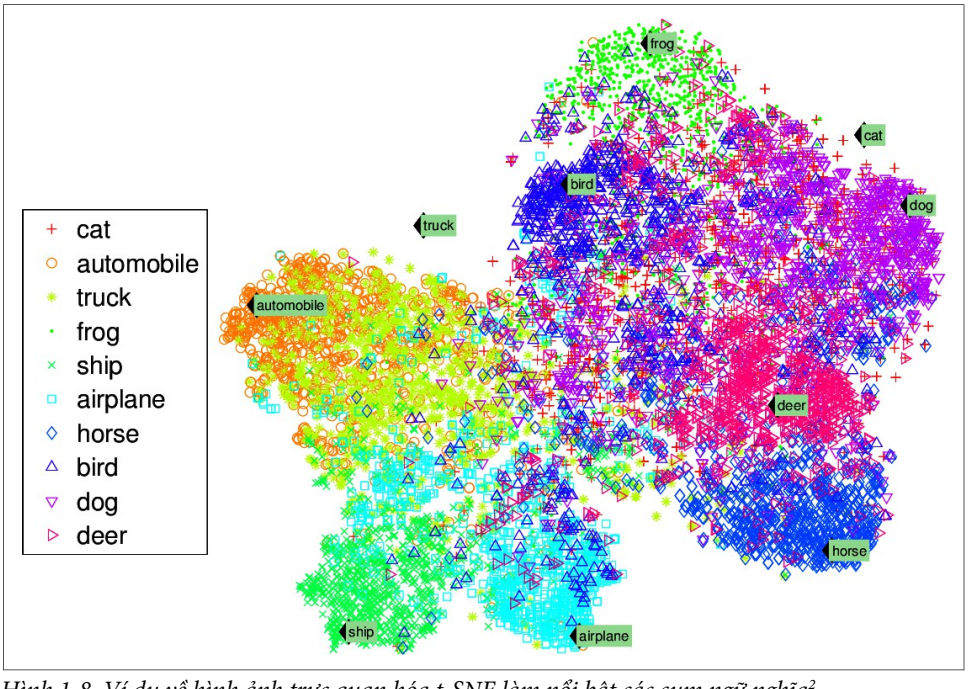
Hình 1-7. Phân cụm

Các thuật toán *trực quan hóa* cũng là những ví dụ điển hình về học máy không giám sát: bạn cung cấp cho chúng một lượng lớn dữ liệu phức tạp và không được gắn nhãn, và chúng sẽ tạo ra một biểu diễn 2D hoặc 3D của dữ liệu đó, có thể dễ dàng được vẽ trên đồ thị (Hình 1-8). Các thuật toán này cố gắng giữ lại càng nhiều cấu trúc càng tốt (ví dụ: cố gắng giữ cho các cụm riêng biệt trong không gian đầu vào không chồng chéo lên nhau trong hình ảnh hóa) để bạn có thể hiểu cách dữ liệu được tổ chức và có thể xác định các mẫu không ngờ tới.

Một tác vụ liên quan là *giảm chiều*, trong đó mục tiêu là đơn giản hóa dữ liệu mà không làm mất quá nhiều thông tin. Một cách để thực hiện điều này là gộp nhiều đặc trưng có tương quan với nhau thành một. Ví dụ, số dặm đã chạy của một chiếc xe có thể có tương quan mạnh với tuổi thọ của nó, do đó thuật toán giảm chiều sẽ gộp chúng thành một đặc trưng đại diện cho mức độ hao mòn của xe. Điều này được gọi là *trích xuất đặc trưng*.



Thường thì bạn nên thử giảm số chiều trong dữ liệu huấn luyện bằng thuật toán giảm chiều trước khi đưa dữ liệu đó vào một thuật toán học máy khác (chẳng hạn như thuật toán học có giám sát). Quá trình này sẽ chạy nhanh hơn nhiều, dữ liệu sẽ chiếm ít dung lượng đĩa và bộ nhớ hơn, và trong một số trường hợp, hiệu suất cũng có thể tốt hơn.

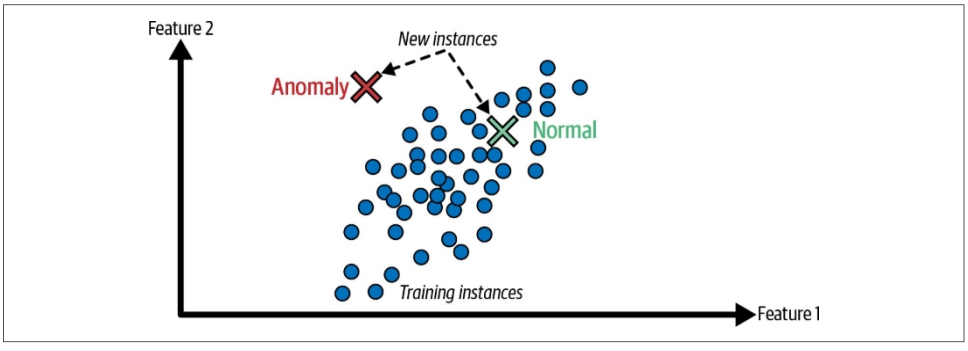


Hình 1-8. Ví dụ về hình ảnh trực quan hóa t-SNE làm nổi bật các cụm ngữ nghĩa[2]

Một tác vụ không giám sát quan trọng khác là *phát hiện dị thường* — ví dụ như phát hiện các giao dịch thẻ tín dụng bất thường để ngăn chặn gian lận, phát hiện các lỗi sản xuất, hoặc tự động loại bỏ các giá trị ngoại lệ khỏi tập dữ liệu trước khi đưa vào một thuật toán học máy khác. Trong quá trình huấn luyện, hệ thống chủ yếu được tiếp xúc với các trường hợp bình thường, do đó nó học cách nhận diện chúng; sau đó, khi gặp một trường hợp mới, hệ thống có thể xác định xem nó có giống với các trường hợp bình thường hay có khả năng là một trường hợp bất thường (xem Hình 1-9). Các đặc trưng có thể bao gồm khoảng cách từ nhà, thời gian trong ngày, ngày trong tuần, số tiền rút, danh mục thương nhân, tần suất giao dịch, v.v. Một tác vụ rất tương tự là *phát hiện sự mới lạ*: nó nhằm phát hiện các trường hợp mới trông khác biệt so với tất cả các trường hợp trong tập huấn luyện. Điều này đòi hỏi phải có một tập huấn luyện rất "sạch", không chứa bất kỳ trường hợp nào mà bạn muốn thuật toán phát hiện. Ví dụ, nếu bạn có hàng nghìn bức ảnh về chó, và 1% trong số đó là chó Chihuahua, thì thuật toán phát hiện sự mới lạ không nên coi các bức ảnh mới về chó Chihuahua là sự mới lạ. Ngược lại, các thuật toán phát hiện bất thường có thể coi những con chó này là quá hiếm và quá khác biệt

2 Hãy chú ý cách các loài động vật được phân tách khá rõ ràng khỏi các phương tiện giao thông và cách ngựa nằm gần hươu nhưng lại cách xa
so với các loài chim. Hình ảnh được sao chép với sự cho phép của Richard Socher et al., "Zero-Shot Learning Through Cross-Modal Transfer", *Kỷ yếu Hội nghị Quốc tế lần thứ 26 về Hệ thống Xử lý Thông tin Thần kinh* 1 (2013): 935–943.

so với những con chó khác đến mức chúng có khả năng sẽ phân loại chúng là những trường hợp bất thường (không có ý xúc phạm giống chó Chihuahua).

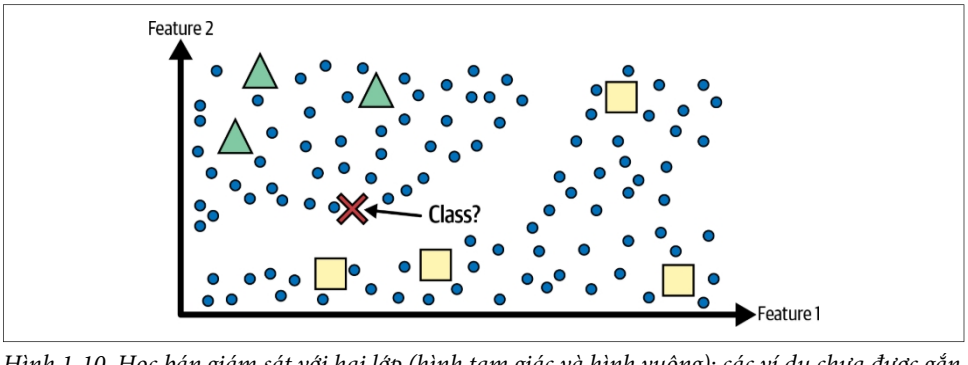


Hình 1-9. Phát hiện dị thường

Cuối cùng, một nhiệm vụ không giám sát phổ biến khác là *học quy tắc liên kết*, trong đó mục tiêu là khai thác lượng lớn dữ liệu để khám phá các mối quan hệ thú vị giữa các thuộc tính. Ví dụ, giả sử bạn sở hữu một siêu thị. Áp dụng quy tắc liên kết vào nhật ký bán hàng có thể cho thấy những người mua nước sốt nướng và khoai tây chiên cũng có xu hướng mua thịt bò. Do đó, bạn có thể muốn đặt các mặt hàng này gần nhau.

Học bán giám sát

Vì việc gắn nhãn dữ liệu thường tốn thời gian và chi phí, bạn thường sẽ có rất nhiều mẫu dữ liệu chưa được gắn nhãn và ít mẫu đã được gắn nhãn. Một số thuật toán có thể xử lý dữ liệu được gắn nhãn một phần. Điều này được gọi là *học bán giám sát* (Hình 1-10).



Hình 1-10. Học bán giám sát với hai lớp (hình tam giác và hình vuông): các ví dụ chưa được gắn nhãn (hình tròn) giúp phân loại một trường hợp mới (dấu thập) vào lớp hình tam giác thay vì lớp hình vuông, mặc dù nó gần với các hình vuông đã được gắn nhãn hơn

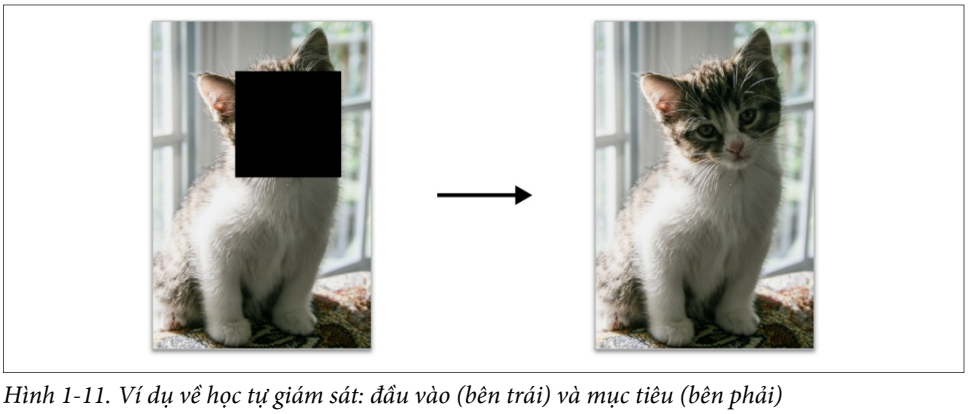
Một số dịch vụ lưu trữ ảnh, chẳng hạn như Google Photos, là những ví dụ điển hình về điều này. Khi bạn tải lên tất cả ảnh gia đình lên dịch vụ này, nó sẽ tự động nhận diện rằng cùng một người A xuất hiện trong các ảnh số 1, 5 và 11, trong khi một người khác B xuất hiện trong các ảnh số 2, 5 và 7. Đây là phần học không giám sát của thuật toán (phân cụm). Giờ đây, tất cả những gì hệ thống cần là bạn chỉ ra những người này là ai. Chỉ cần thêm một nhãn cho mỗi người[3] là hệ thống có thể đặt tên cho mọi người trong mỗi bức ảnh, điều này rất hữu ích cho việc tìm kiếm ảnh.

Hầu hết các thuật toán học bán có giám sát là sự kết hợp giữa các thuật toán không có giám sát và có giám sát. Ví dụ, một thuật toán phân cụm có thể được sử dụng để nhóm các trường hợp tương tự lại với nhau, và sau đó mỗi trường hợp chưa được gắn nhãn có thể được gắn nhãn bằng nhãn phổ biến nhất trong cụm của nó. Khi toàn bộ tập dữ liệu đã được gắn nhãn, có thể sử dụng bất kỳ thuật toán học có giám sát nào.

Học tự giám sát

Một phương pháp khác trong học máy là tạo ra một tập dữ liệu được gắn nhãn hoàn toàn từ một tập dữ liệu hoàn toàn chưa được gắn nhãn. Một lần nữa, sau khi toàn bộ tập dữ liệu được gắn nhãn, có thể sử dụng bất kỳ thuật toán học có giám sát nào. Phương pháp này được gọi là *học tự giám sát*.

Ví dụ: nếu bạn có một tập dữ liệu lớn gồm các hình ảnh chưa được gắn nhãn, bạn có thể che ngẫu nhiên một phần nhỏ của mỗi hình ảnh, sau đó huấn luyện mô hình để khôi phục hình ảnh gốc (Hình 1-11). Trong quá trình huấn luyện, các hình ảnh bị che được sử dụng làm đầu vào cho mô hình, còn các hình ảnh gốc được sử dụng làm nhãn.



Hình 1-11. Ví dụ về học tự giám sát: đầu vào (bên trái) và mục tiêu (bên phải)

3 Đó là khi hệ thống hoạt động hoàn hảo. Trên thực tế, hệ thống thường tạo ra một vài cụm cho mỗi người, và đôi khi nhầm lẫn hai người trông giống nhau, do đó bạn có thể cần cung cấp một vài nhãn cho mỗi người và dọn dẹp thủ công một số cụm.

Mô hình thu được có thể khá hữu ích ngay từ bản thân nó — ví dụ như để sửa chữa hình ảnh bị hư hỏng hoặc xóa các đối tượng không mong muốn khỏi hình ảnh. Tuy nhiên, phần lớn thời gian, mô hình được huấn luyện bằng học tự giám sát không phải là mục tiêu cuối cùng. Bạn thường sẽ muốn điều chỉnh và tinh chỉnh mô hình cho một tác vụ hơi khác — một tác vụ mà bạn thực sự quan tâm.

Ví dụ, giả sử điều bạn thực sự muốn là có một mô hình phân loại thú cưng: khi được cung cấp hình ảnh của bất kỳ con thú cưng nào, mô hình sẽ cho bạn biết nó thuộc loài nào. Nếu bạn có một tập dữ liệu lớn gồm các bức ảnh thú cưng chưa được gắn nhãn, bạn có thể bắt đầu bằng cách huấn luyện một mô hình sửa chữa hình ảnh bằng phương pháp học tự giám sát. Khi mô hình hoạt động tốt, nó sẽ có khả năng phân biệt các loài thú cưng khác nhau: khi sửa chữa hình ảnh của một con mèo có khuôn mặt bị che khuất, nó phải biết không được thêm khuôn mặt của một con chó vào đó. Giả sử kiến trúc mô hình của bạn cho phép điều này (và hầu hết các kiến trúc mạng nơ-ron đều cho phép), thì bạn có thể điều chỉnh mô hình để nó dự đoán loài vật nuôi thay vì phục hồi hình ảnh. Bước cuối cùng là tinh chỉnh mô hình trên một tập dữ liệu có nhãn: mô hình đã biết mèo, chó và các loài vật nuôi khác trông như thế nào, nên bước này chỉ cần thiết để mô hình có thể học được mối tương ứng giữa các loài mà nó đã biết và các nhãn mà chúng ta mong đợi từ nó.



Việc chuyển giao kiến thức từ một tác vụ sang tác vụ khác được gọi là *học chuyển giao (transfer learning)*, và đây là một trong những kỹ thuật quan trọng nhất trong lĩnh vực học máy hiện nay, đặc biệt khi sử dụng *các mạng nơ-ron sâu* (tức là các mạng nơ-ron bao gồm nhiều lớp nơ-ron). Chúng ta sẽ thảo luận chi tiết về vấn đề này trong Phần II.

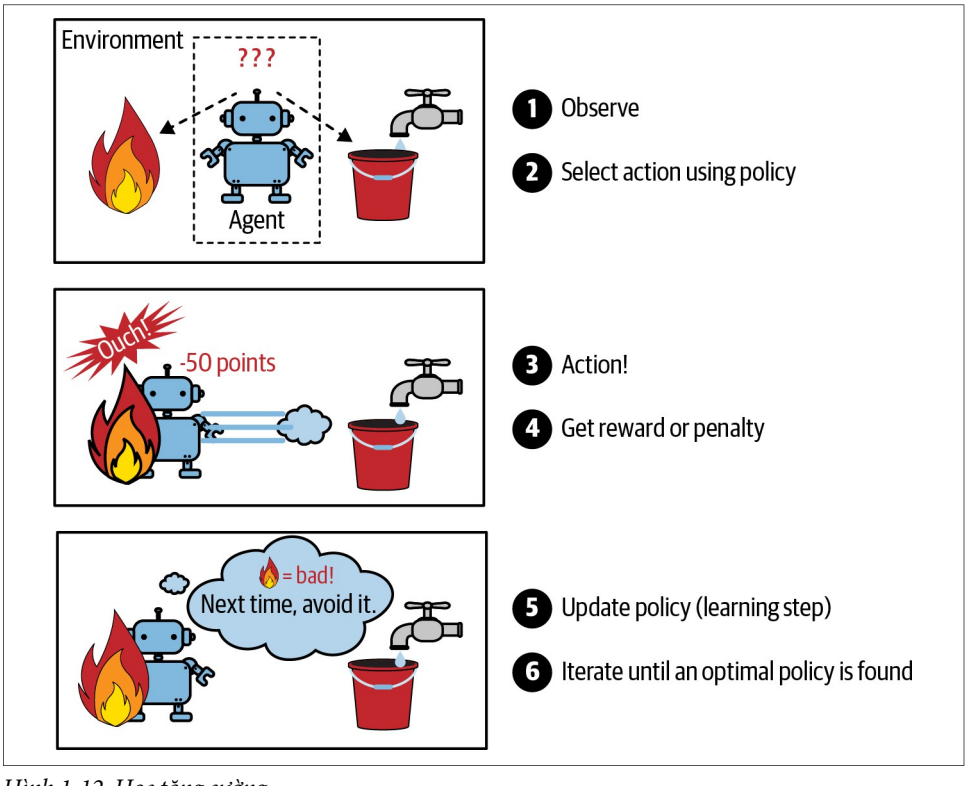
Như chúng ta sẽ thấy trong Chương 15, các mô hình ngôn ngữ lớn (LLMs) được huấn luyện theo cách rất tương tự, bằng cách che đi các từ ngẫu nhiên trong một tập dữ liệu văn bản khổng lồ và huấn luyện mô hình để dự đoán các từ bị thiếu. Mô hình được huấn luyện sẵn quy mô lớn này sau đó có thể được tinh chỉnh cho các ứng dụng khác nhau, từ phân tích cảm xúc đến chatbot.

Một số người coi học tự giám sát là một phần của học không giám sát, vì nó xử lý các tập dữ liệu hoàn toàn không có nhãn. Tuy nhiên, học tự giám sát sử dụng các nhãn (được tạo ra) trong quá trình huấn luyện, nên về mặt này, nó gần với học có giám sát hơn. Và thuật ngữ "học không giám sát" thường được sử dụng khi xử lý các tác vụ như phân cụm, giảm chiều hoặc phát hiện dị thường, trong khi học tự giám sát tập trung vào các tác vụ tương tự như học có giám sát: chủ yếu là phân loại và hồi quy. Tóm lại, tốt nhất là nên coi học tự giám sát như một thể loại riêng biệt.

Học tăng cường

Học tăng cường là một khái niệm hoàn toàn khác biệt. Hệ thống học tập, được gọi là *tác nhân (agent)* trong bối cảnh này, có thể quan sát môi trường, lựa chọn và thực hiện các hành động, đồng thời nhận được phần *thưởng* (hoặc *hình phạt* dưới dạng phần thưởng âm, như minh họa trong Hình 1-12). Sau đó, nó phải tự học để xác định chiến lược tối ưu, được gọi là *chính sách (policy)*, nhằm đạt được

nhận được phần thưởng tối đa theo thời gian. Chính sách xác định hành động mà tác nhân nên chọn khi ở trong một tình huống nhất định.



Hình 1-12. Học tăng cường

Ví dụ, nhiều robot áp dụng các thuật toán học tăng cường để học cách đi lại. Chương trình AlphaGo của DeepMind cũng là một ví dụ điển hình về học tăng cường: nó đã gây chấn động vào tháng 5 năm 2017 khi đánh bại Ke Jie, kỳ thủ số một thế giới lúc bấy giờ, trong trò chơi cờ vây. Nó đã học được chính sách chiến thắng của mình bằng cách phân tích hàng triệu ván cờ, sau đó tự chơi nhiều ván cờ với chính mình. Lưu ý rằng quá trình học đã bị tắt trong các ván đấu với nhà vô địch; AlphaGo chỉ áp dụng chiến lược mà nó đã học được. Như bạn sẽ thấy trong phần tiếp theo, đây được gọi là *học ngoại tuyến*.

Học theo lô so với học trực tuyến

Một tiêu chí khác được sử dụng để phân loại các hệ thống học máy là liệu hệ thống có thể học dần dần từ một luồng dữ liệu đầu vào hay không. Ví dụ, rừng ngẫu nhiên (xem Chương 6) chỉ có thể được huấn luyện từ đầu trên toàn bộ tập dữ liệu — đây được gọi là

học theo lô — trong khi các mô hình khác có thể được huấn luyện từng lô dữ liệu một, ví dụ như sử dụng *phương pháp gradient descent* (xem Chương 4) — đây được gọi là học trực tuyến.

Học theo lô

Trong *học theo lô*, hệ thống phải được huấn luyện bằng toàn bộ dữ liệu có sẵn. Quá trình này thường tốn nhiều thời gian và tài nguyên tính toán, do đó thường được thực hiện ngoại tuyến. Đầu tiên hệ thống được huấn luyện, sau đó được triển khai vào môi trường sản xuất và chạy mà không học thêm nữa; nó chỉ áp dụng những gì đã học được. Điều này được gọi là *học ngoại tuyến*.

Thật không may, hiệu suất của mô hình có xu hướng suy giảm dần theo thời gian, đơn giản vì thế giới tiếp tục phát triển trong khi mô hình vẫn giữ nguyên. Hiện tượng này thường được gọi là *sự trôi dạt dữ liệu* (hoặc *sự suy giảm mô hình*). Giải pháp là định kỳ huấn luyện lại mô hình trên dữ liệu cập nhật. Tần suất cần thực hiện việc này phụ thuộc vào trường hợp sử dụng: nếu mô hình phân loại hình ảnh mèo và chó, hiệu suất của nó sẽ suy giảm rất chậm, nhưng nếu mô hình xử lý các hệ thống thay đổi nhanh chóng, ví dụ như đưa ra dự đoán trên thị trường tài chính, thì hiệu suất có khả năng suy giảm khá nhanh.



Ngay cả một mô hình được huấn luyện để phân loại hình ảnh mèo và chó cũng có thể cần được huấn luyện lại thường xuyên, không phải vì mèo và chó sẽ đột biến chỉ trong một đêm, mà vì máy ảnh liên tục thay đổi, cùng với định dạng hình ảnh, độ sắc nét, độ sáng và tỷ lệ kích thước. Hơn nữa, năm sau mọi người có thể yêu thích những giống vật nuôi khác, hoặc họ có thể quyết định đội những chiếc mũ nhỏ xíu cho thú cưng của mình — ai mà biết được?

Nếu bạn muốn hệ thống học theo lô nhận biết dữ liệu mới (chẳng hạn như một loại spam mới), bạn cần huấn luyện một phiên bản mới của hệ thống từ đầu trên toàn bộ tập dữ liệu (không chỉ dữ liệu mới mà cả dữ liệu cũ), sau đó thay thế mô hình cũ bằng mô hình mới. May mắn thay, toàn bộ quy trình huấn luyện, đánh giá và triển khai hệ thống học máy có thể được tự động hóa (như chúng ta đã thấy trong Hình 1-3), do đó ngay cả hệ thống học theo lô cũng có thể thích ứng với sự thay đổi. Chỉ cần cập nhật dữ liệu và huấn luyện một phiên bản mới của hệ thống từ đầu mỗi khi cần thiết.

Giải pháp này đơn giản và thường hoạt động tốt, nhưng việc huấn luyện trên toàn bộ tập dữ liệu có thể mất nhiều giờ, do đó bạn thường chỉ huấn luyện hệ thống mới mỗi 24 giờ hoặc thậm chí chỉ hàng tuần. Nếu hệ thống của bạn cần thích ứng với dữ liệu thay đổi nhanh chóng (ví dụ: để dự đoán giá cổ phiếu), thì bạn cần một giải pháp phản ứng nhanh hơn.

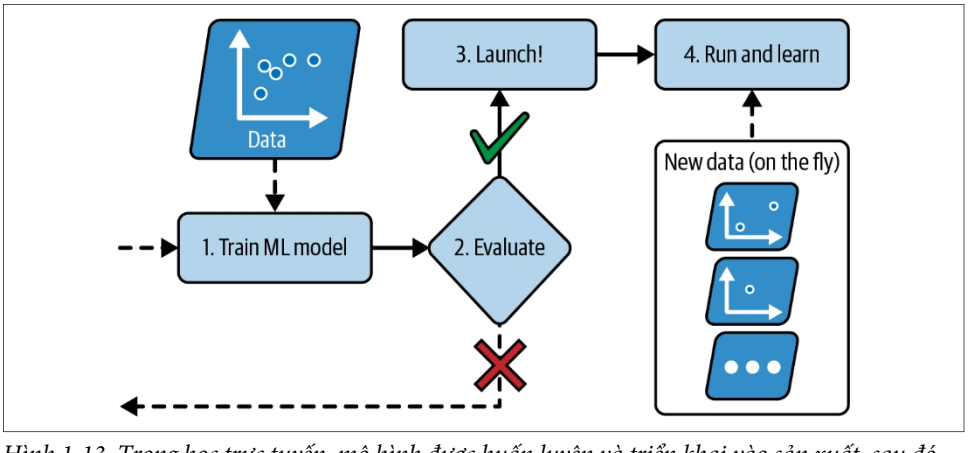
Ngoài ra, việc huấn luyện trên toàn bộ tập dữ liệu đòi hỏi rất nhiều tài nguyên tính toán (CPU, bộ nhớ, dung lượng đĩa, I/O đĩa, I/O mạng, v.v.). Nếu bạn có lượng dữ liệu lớn và tự động hóa hệ thống để huấn luyện lại từ đầu mỗi ngày, điều này sẽ tiêu tốn của bạn rất nhiều chi phí. Nếu lượng dữ liệu khổng lồ và hệ thống của bạn phải luôn được cập nhật, thậm chí có thể không thể áp dụng học theo lô.

Cuối cùng, nếu hệ thống của bạn cần có khả năng học tập tự chủ và có tài nguyên hạn chế (ví dụ: một ứng dụng trên điện thoại thông minh hoặc một xe thám hiểm trên sao Hỏa), thì việc mang theo lượng lớn dữ liệu huấn luyện và tiêu tốn nhiều tài nguyên để huấn luyện hàng giờ mỗi ngày sẽ là một trở ngại lớn.

Một lựa chọn tốt hơn trong tất cả các tình huống này là sử dụng các thuật toán có khả năng học tập từng bước.

Học trực tuyến

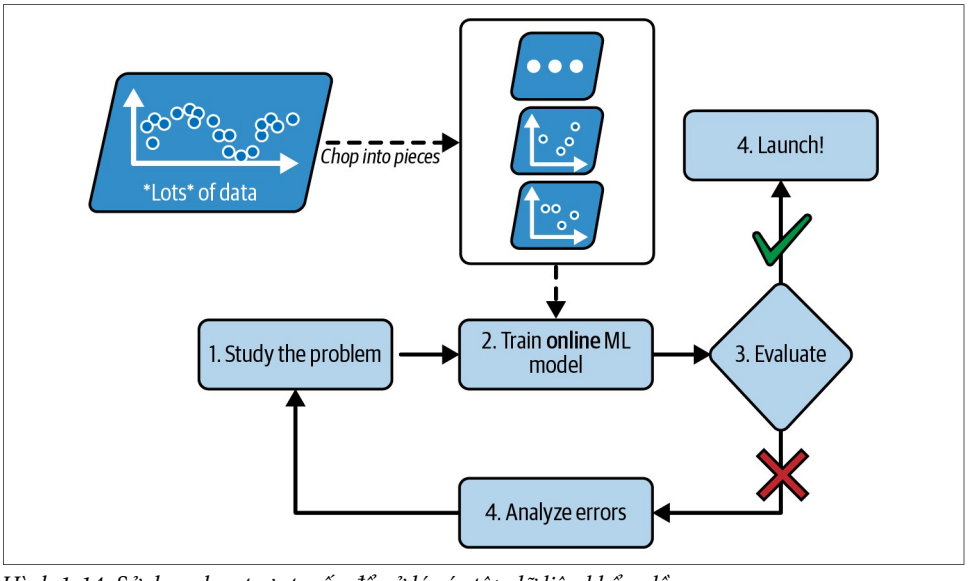
Trong *học trực tuyến*, bạn huấn luyện hệ thống theo từng bước bằng cách cung cấp các mẫu dữ liệu theo thứ tự, hoặc từng mẫu một hoặc theo các nhóm nhỏ gọi là *mini-batch*. Mỗi bước học diễn ra nhanh chóng và tiết kiệm tài nguyên, do đó hệ thống có thể học từ dữ liệu mới ngay lập tức khi dữ liệu đó được cung cấp (xem Hình 1-13). Thuật toán học trực tuyến phổ biến nhất hiện nay là gradient descent, nhưng cũng có một số thuật toán khác.



Hình 1-13. Trong học trực tuyến, mô hình được huấn luyện và triển khai vào sản xuất, sau đó tiếp tục học khi dữ liệu mới được đưa vào

Học trực tuyến rất hữu ích cho các hệ thống cần thích ứng với sự thay đổi cực kỳ nhanh chóng (ví dụ: để phát hiện các mẫu mới trên thị trường chứng khoán). Đây cũng là một lựa chọn tốt nếu bạn có tài nguyên tính toán hạn chế; ví dụ, nếu mô hình được huấn luyện trên thiết bị di động.

Quan trọng nhất, các thuật toán học trực tuyến có thể được sử dụng để huấn luyện mô hình trên các tập dữ liệu khổng lồ mà bộ nhớ của một máy tính không thể chứa hết (điều này được gọi là học *ngoài bộ nhớ*). Thuật toán sẽ tải một phần dữ liệu, thực hiện bước huấn luyện trên phần dữ liệu đó, và lặp lại quy trình cho đến khi đã xử lý hết toàn bộ dữ liệu (xem Hình 1-14).



Hình 1-14. Sử dụng học trực tuyến để xử lý các tập dữ liệu khổng lồ

Một tham số quan trọng của các hệ thống học trực tuyến là tốc độ thích ứng với dữ liệu thay đổi: đây được gọi là *tốc độ học*. Nếu bạn đặt tốc độ học cao, hệ thống sẽ nhanh chóng thích ứng với dữ liệu mới, nhưng cũng có xu hướng nhanh chóng quên đi dữ liệu cũ: hiện tượng này được gọi là *hiện tượng quên thảm khốc* (hoặc *hiện tượng can thiệp thảm khốc*). Bạn không muốn bộ lọc spam chỉ đánh dấu các loại spam mới nhất mà nó được hiển thị! Ngược lại, nếu bạn đặt tốc độ học thấp, hệ thống sẽ có độ trễ cao hơn; tức là, nó sẽ học chậm hơn, nhưng cũng ít nhạy cảm hơn với nhiễu trong dữ liệu mới hoặc với các chuỗi điểm dữ liệu không đại diện (các giá trị ngoại lệ).



Học ngoài bộ nhớ (out-of-core learning) thường được thực hiện ngoại tuyến (tức là không trên hệ thống đang hoạt động), do đó thuật ngữ *"học trực tuyến" (online learning)* có thể gây nhầm lẫn. Hãy coi đó là *học tăng dần*. Hơn nữa, các lô nhỏ (mini-batches) thường chỉ được gọi là "lô" (*batches*), nên thuật ngữ *"học theo lô" (batch learning)* cũng gây nhầm lẫn. Hãy coi đó là việc học từ đầu trên toàn bộ tập dữ liệu.

Một thách thức lớn với học trực tuyến là nếu dữ liệu xấu được đưa vào hệ thống, hiệu suất của hệ thống sẽ giảm sút, có thể rất nhanh (tùy thuộc vào chất lượng dữ liệu và tốc độ học). Nếu đó là hệ thống đang hoạt động, khách hàng của bạn sẽ nhận ra điều này. Ví dụ, dữ liệu xấu có thể xuất phát từ một lỗi (ví dụ: cảm biến hỏng trên robot), hoặc có thể do ai đó cố tình lạm dụng hệ thống (ví dụ: spam công cụ tìm kiếm để cố gắng xếp hạng cao trong kết quả tìm kiếm). Để giảm thiểu rủi ro này, bạn cần theo dõi hệ thống chặt chẽ và tắt chế độ học ngay lập tức (và có thể khôi phục về trạng thái hoạt động trước đó) nếu phát hiện hiệu suất giảm sút. Bạn cũng có thể muốn theo dõi dữ liệu đầu vào

và phản ứng với dữ liệu bất thường; ví dụ: sử dụng thuật toán phát hiện dị thường (xem Chương 8).

Học dựa trên trường hợp so với học dựa trên mô hình

Một cách khác để phân loại các hệ thống học máy là dựa trên khả năng *tổng quát hóa* của chúng. Hầu hết các tác vụ học máy đều liên quan đến việc đưa ra dự đoán. Điều này có nghĩa là, với một số ví dụ huấn luyện nhất định, hệ thống cần có khả năng đưa ra dự đoán chính xác (tổng quát hóa) cho các ví dụ mà nó chưa từng gặp trước đây. Việc đạt được chỉ số hiệu suất tốt trên dữ liệu huấn luyện là tốt, nhưng chưa đủ; mục tiêu thực sự là đạt hiệu suất tốt trên các trường hợp mới.

Có hai phương pháp chính để tổng quát hóa: học dựa trên trường hợp và học dựa trên mô hình.

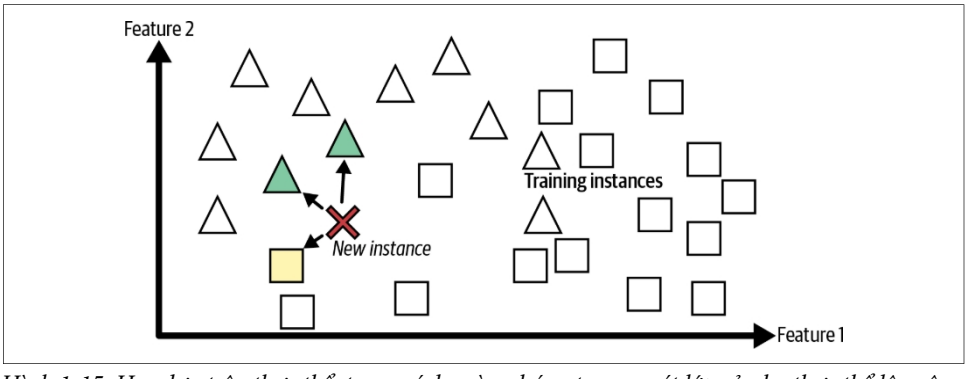
Học dựa trên trường hợp

Có lẽ hình thức học đơn giản nhất là học thuộc lòng. Nếu bạn tạo bộ lọc thư rác theo cách này, nó sẽ chỉ đánh dấu tất cả các email giống hệt với những email đã được người dùng đánh dấu trước đó — không phải là giải pháp tồi nhất, nhưng chắc chắn không phải là tốt nhất.

Thay vì chỉ đánh dấu các email trùng khớp với các email spam đã biết, bộ lọc spam của bạn có thể được lập trình để cũng đánh dấu các email rất tương tự với các email spam đã biết. Điều này đòi hỏi một *thước đo độ tương tự* giữa hai email. Một thước đo độ tương tự (rất cơ bản) giữa hai email có thể là đếm số từ chung giữa chúng. Hệ thống sẽ đánh dấu một email là spam nếu nó có nhiều từ chung với một email spam đã biết.

Điều này được gọi *là học dựa trên thực thể*: hệ thống ghi nhớ các ví dụ, sau đó suy rộng ra các trường hợp mới bằng cách sử dụng thước đo độ tương đồng để so sánh chúng với các ví dụ đã học (hoặc một tập con của chúng). Ví dụ, trong Hình 1-15, thực thể mới sẽ được phân loại là tam giác vì phần lớn các thực thể tương đồng nhất thuộc về lớp đó.

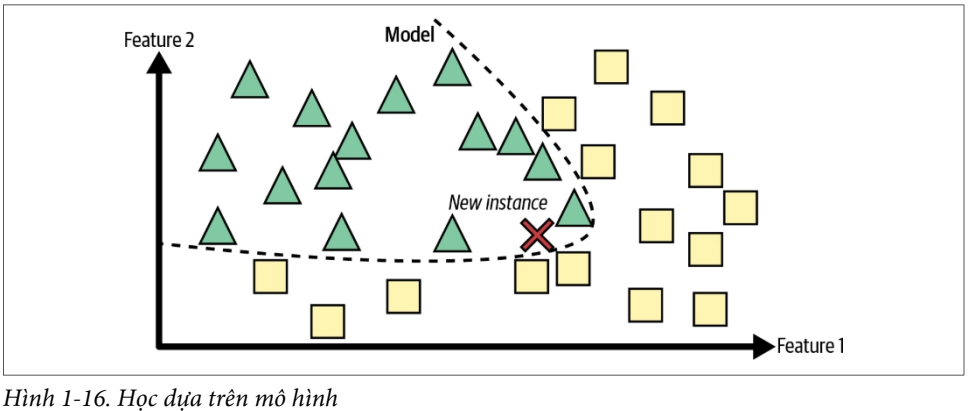
Học dựa trên thực thể thường phát huy hiệu quả với các tập dữ liệu nhỏ, đặc biệt khi dữ liệu liên tục thay đổi, nhưng phương pháp này không mở rộng quy mô tốt: nó yêu cầu triển khai toàn bộ bản sao của tập huấn luyện sang môi trường sản xuất; việc dự đoán đòi hỏi phải tìm kiếm các thực thể tương tự, điều này có thể khá chậm; và nó không hoạt động hiệu quả với dữ liệu đa chiều như hình ảnh.



Hình 1-15. Học dựa trên thực thể: trong ví dụ này, chúng ta xem xét lớp của ba thực thể lân cận gần nhất trong tập huấn luyện

Học dựa trên mô hình và quy trình làm việc điển hình của học máy

Một cách khác để khái quát hóa từ một tập hợp các ví dụ là xây dựng một mô hình từ các ví dụ này và sau đó sử dụng mô hình đó để đưa ra *dự đoán*. Đây được gọi là *học dựa trên mô hình* (Hình 1-16).



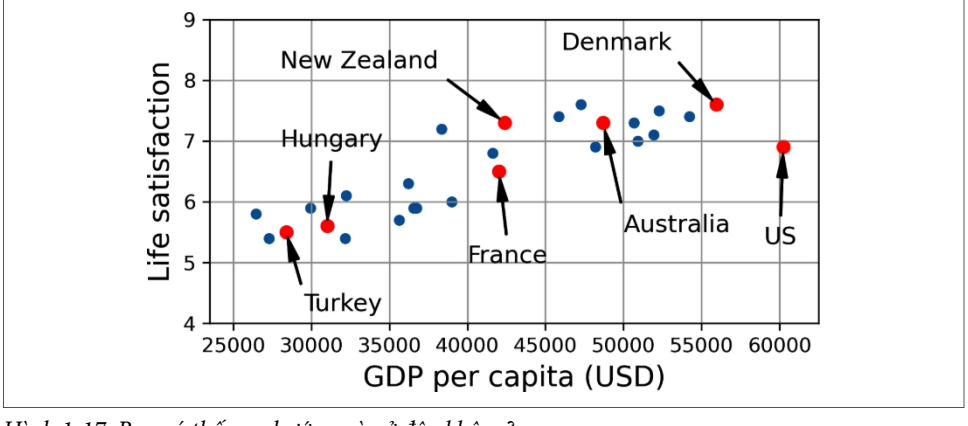
Hình 1-16. Học dựa trên mô hình

Ví dụ, giả sử bạn muốn biết liệu tiền có làm cho con người hạnh phúc hay không, nên bạn tải xuống dữ liệu Chỉ số Cuộc sống Tốt đẹp hơn (Better Life Index) từ trang web của OECD và số liệu thống kê của Ngân hàng Thế giới về tổng sản phẩm quốc nội (GDP) bình quân đầu người. Sau đó, bạn ghép các bảng lại với nhau và sắp xếp theo GDP bình quân đầu người. Bảng 1-1 cho thấy một đoạn trích từ kết quả bạn thu được.

Bảng 1-1. Tiền có làm cho con người hạnh phúc hơn không?

Quốc gia	GDP bình quân đầu người (USD)	Mức độ hài lòng với cuộc sống
Thổ Nhĩ Kỳ	28.384	5,5
Hungary	31.008	5,6
Pháp	42.026	6,5
Hoa Kỳ	60.236	6,9
New Zealand	42.404	7,3
Úc	48.698	7,3
Đan Mạch	55.938	7,6

Hãy vẽ biểu đồ dữ liệu cho các quốc gia này (Hình 1-17).



Hình 1-17. Bạn có thấy xu hướng nào ở đây không?

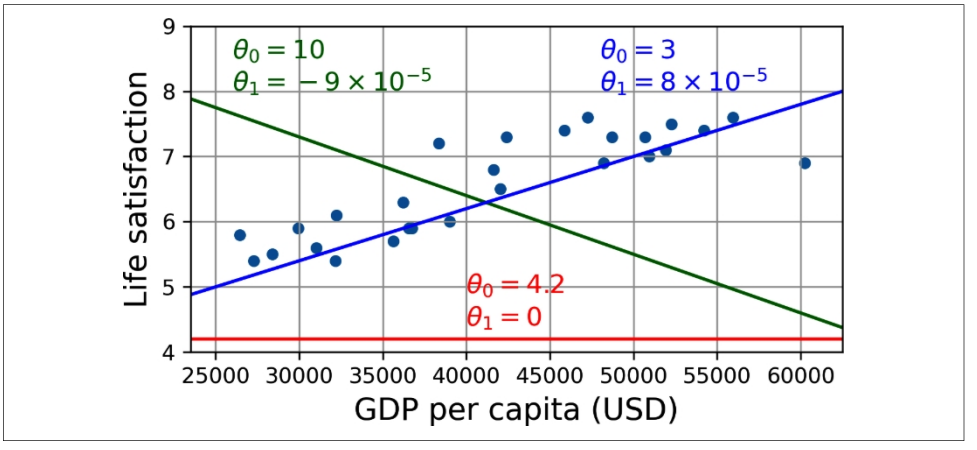
Có vẻ như có một xu hướng ở đây! Mặc dù dữ liệu có *nhiễu* (tức là một phần ngẫu nhiên), nhưng có vẻ như mức độ hài lòng với cuộc sống tăng lên gần như tuyến tính khi GDP bình quân đầu người của quốc gia đó tăng lên. Vì vậy, bạn quyết định mô hình hóa mức độ hài lòng với cuộc sống như một hàm tuyến tính của GDP bình quân đầu người (bạn giả định rằng bất kỳ sự sai lệch nào so với đường thẳng đó chỉ là nhiễu ngẫu nhiên). Bước này được gọi là *lựa chọn mô hình*: bạn đã chọn một *mô hình tuyến tính* về mức độ hài lòng với cuộc sống chỉ với một thuộc tính duy nhất, đó là GDP bình quân đầu người (Phương trình 1-1).

Phương trình 1-1. Một mô hình tuyến tính đơn giản

$$\text{mức_độ_hài_lòng_với_cuộc_sống} = \theta_0 + \theta_1 \times \text{GDP_bình_quân_đầu_người}$$

Mô hình này có hai *tham số mô hình*, θ_0 và θ_1.[4] Bằng cách điều chỉnh các tham số này, bạn có thể làm cho mô hình của mình biểu diễn bất kỳ hàm tuyến tính nào, như được thể hiện trong Hình 1-18.

4 Theo quy ước, chữ cái Hy Lạp θ (theta) thường được sử dụng để biểu thị các tham số mô hình.



Hình 1-18. Một số mô hình tuyến tính có thể có

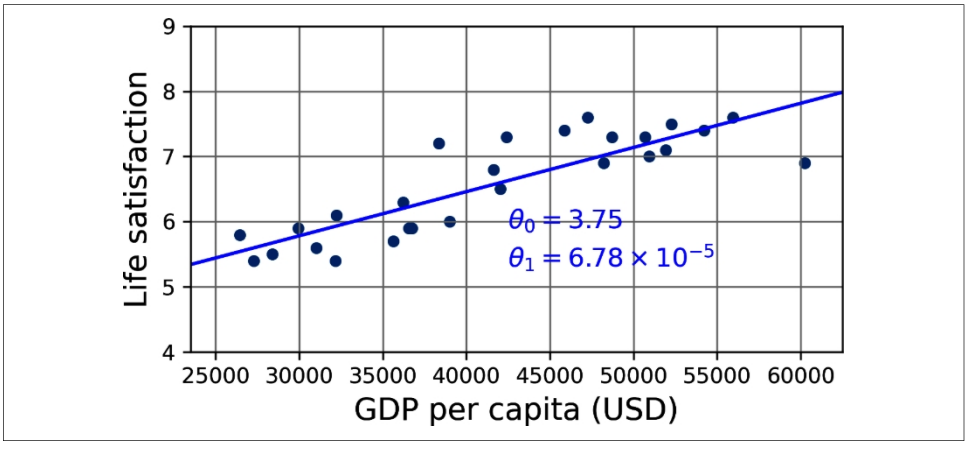
Trước khi có thể sử dụng mô hình, bạn cần xác định các giá trị tham số $\theta_0$và θ_1. Làm thế nào để biết các giá trị nào sẽ giúp mô hình hoạt động tốt nhất? Để trả lời câu hỏi này, bạn cần xác định một thước đo hiệu suất. Bạn có thể định nghĩa một *hàm tiện ích* (hoặc *hàm phù hợp*) để đo lường mức độ *tốt* của mô hình, hoặc định nghĩa một *hàm chi phí* (còn gọi là *hàm mất mát*) để đo lường mức độ *xấu* của nó. Đối với các bài toán hồi quy tuyến tính, người ta thường sử dụng hàm chi phí để đo khoảng cách giữa các dự đoán của mô hình tuyến tính và các mẫu huấn luyện; mục tiêu là giảm thiểu khoảng cách này.

Đây chính là lúc thuật toán hồi quy tuyến tính phát huy tác dụng: bạn cung cấp các mẫu huấn luyện cho thuật toán, và nó sẽ tìm ra các tham số giúp mô hình tuyến tính phù hợp nhất với dữ liệu của bạn. Quá trình này được gọi là *huấn luyện* mô hình. Trong trường hợp của chúng ta, thuật toán xác định rằng các giá trị tham số tối ưu là $\theta_0 = 3{,}75$ và $\theta_1 = 6{,}78 \times 10\text{^}{-5}$.



Điều gây nhầm lẫn là từ "mô hình" có thể ám chỉ một *loại mô hình* (ví dụ: hồi quy tuyến tính), một *kiến trúc mô hình được xác định đầy đủ* (ví dụ: hồi quy tuyến tính với một đầu vào và một đầu ra), hoặc *mô hình đã được huấn luyện hoàn chỉnh* và sẵn sàng để sử dụng cho việc dự đoán (ví dụ: hồi quy tuyến tính với một đầu vào và một đầu ra, sử dụng $\theta_0 = 3{,}75$ và $\theta_1 = 6{,}78 \times^{10^{-5}}$). Lựa chọn mô hình bao gồm việc chọn loại mô hình và xác định đầy đủ kiến trúc của nó. Huấn luyện mô hình có nghĩa là chạy một thuật toán để tìm các tham số mô hình giúp mô hình phù hợp nhất với dữ liệu huấn luyện, và hy vọng sẽ đưa ra các dự đoán chính xác trên dữ liệu mới.

Giờ đây, mô hình đã phù hợp với dữ liệu huấn luyện một cách sát nhất có thể (đối với mô hình tuyến tính), như bạn có thể thấy trong Hình 1-19.



Hình 1-19. Mô hình tuyến tính phù hợp nhất với dữ liệu huấn luyện

Cuối cùng, bạn đã sẵn sàng chạy mô hình để đưa ra dự đoán. Ví dụ, giả sử bạn muốn biết người dân Puerto Rico hạnh phúc đến mức nào, nhưng dữ liệu của OECD lại không cung cấp câu trả lời. May mắn thay, bạn có thể sử dụng mô hình của mình để đưa ra một dự đoán chính xác: bạn tra cứu GDP bình quân đầu người của Puerto Rico, tìm thấy con số 33.442 USD, sau đó áp dụng mô hình của mình và thấy rằng mức độ hài lòng với cuộc sống có thể vào khoảng $3,75 + 33.442 \times 6,78 \times 10^{-5} = 6,02$.

Để khơi gợi sự tò mò của bạn, Ví dụ 1-1 trình bày mã Python tải dữ liệu, tách các biến đầu vào `X` khỏi nhãn `y`, tạo biểu đồ phân tán để trực quan hóa, sau đó huấn luyện mô hình tuyến tính và đưa ra dự đoán.[5]

Ví dụ 1-1. Huấn luyện và chạy mô hình tuyến tính bằng Scikit-Learn

```
import matplotlib.pyplot as plt import
numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Tải xuống và chuẩn bị dữ liệu
data_root = "https://github.com/ageron/data/raw/main/" lifesat =
pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values y
= lifesat[["Life satisfaction"]].values

# Trực quan hóa dữ liệu
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()
```

5 Không sao nếu bạn chưa hiểu hết mã lệnh này; tôi sẽ giới thiệu Scikit-Learn trong các chương tiếp theo.

```
# Chọn mô hình tuyến tính
model = LinearRegression()

# Huấn luyện mô hình
model.fit(X, y)

# Dự đoán cho Puerto Rico
X_new = [[33_442.8]] # GDP bình quân đầu người của Puerto Rico năm 2020
print(model.predict(X_new)) # hiển thị [[6.01610329]]
```



Nếu bạn sử dụng thuật toán học dựa trên thực thể thay vào đó, bạn sẽ thấy rằng Ba Lan có GDP bình quân đầu người gần nhất với Puerto Rico ($32,238), và vì dữ liệu của OECD cho biết mức độ hài lòng với cuộc sống của người Ba Lan là 6.1, bạn cũng sẽ dự đoán mức độ hài lòng với cuộc sống của Puerto Rico là 6.1. Nếu bạn mở rộng phạm vi quan sát một chút và xem xét hai quốc gia gần nhất tiếp theo, bạn sẽ thấy Bồ Đào Nha có mức độ hài lòng với cuộc sống là 5,4, và Estonia có mức độ hài lòng với cuộc sống là 5,7. Tính trung bình ba giá trị này, bạn sẽ được 5,73, con số này thấp hơn một chút so với dự đoán dựa trên mô hình của bạn. Thuật toán đơn giản này được gọi là hồi quy *k-láng giềng gần nhất* (trong ví dụ này, $k = 3$).

Việc thay thế mô hình hồi quy tuyến tính bằng hồi quy k-láng giềng gần nhất trong đoạn mã trước đó rất đơn giản, chỉ cần thay thế các dòng sau:

```
from sklearn.linear_model import LinearRegression model
= LinearRegression()
```

bằng hai dòng sau:

```
from sklearn.neighbors import KNeighborsRegressor model
= KNeighborsRegressor(n_neighbors=3)
```

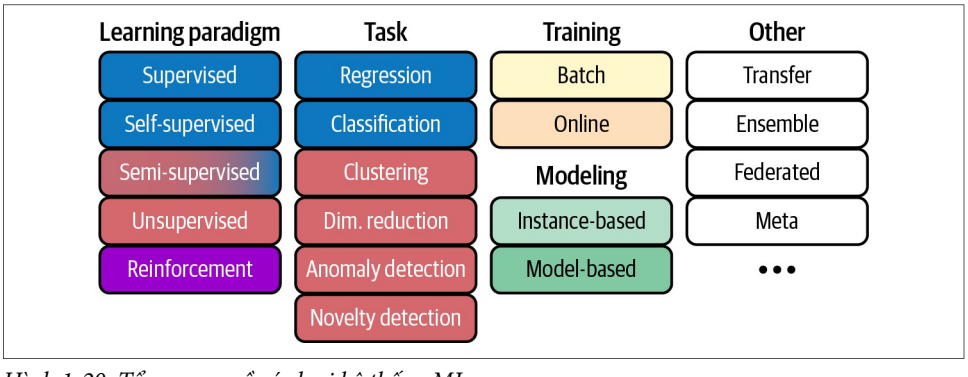
Nếu mọi thứ diễn ra suôn sẻ, mô hình của bạn sẽ đưa ra các dự đoán chính xác. Nếu không, bạn có thể cần sử dụng thêm các thuộc tính (tỷ lệ việc làm, sức khỏe, ô nhiễm không khí, v.v.), thu thập thêm dữ liệu huấn luyện hoặc dữ liệu huấn luyện có chất lượng tốt hơn, hoặc có thể chọn một mô hình mạnh mẽ hơn (ví dụ: mô hình hồi quy đa thức).

Tóm lại:

- Bạn đã nghiên cứu dữ liệu.
- Bạn đã chọn một mô hình.
- Bạn đã huấn luyện mô hình trên dữ liệu huấn luyện (tức là thuật toán học máy đã tìm kiếm các giá trị tham số mô hình sao cho hàm chi phí đạt giá trị nhỏ nhất).
- Cuối cùng, bạn đã áp dụng mô hình để đưa ra dự đoán cho các trường hợp mới (điều này được gọi là *suy luận*), với hy vọng rằng mô hình này sẽ có khả năng tổng quát hóa tốt.

Đây là quy trình điển hình của một dự án học máy. Trong Chương 2, bạn sẽ trải nghiệm quy trình này trực tiếp bằng cách thực hiện một dự án từ đầu đến cuối.

Chúng ta đã thảo luận về khá nhiều loại hệ thống học máy, nhưng lĩnh vực này còn nhiều hơn thế! Ví dụ, *học tập tổng hợp (ensemble learning)* liên quan đến việc huấn luyện nhiều mô hình và kết hợp các dự đoán riêng lẻ của chúng thành các dự đoán được cải thiện (xem Chương 6); *học liên kết (federated learning)* là phương pháp phi tập trung, trong đó các mô hình được huấn luyện trên nhiều thiết bị (ví dụ: điện thoại thông minh) và được điều chỉnh cho từng người dùng mà không trao đổi dữ liệu thô, từ đó bảo vệ quyền riêng tư của người dùng; học siêu *(meta-learning)* là phương pháp "học cách học", trong đó các mô hình học cách thực hiện các tác vụ mới một cách nhanh chóng với lượng dữ liệu tối thiểu. Và danh sách này còn tiếp tục! Hình 1-20 tóm tắt các phân loại khác nhau của các hệ thống học máy mà chúng ta đã thảo luận cho đến nay.



Hình 1-20. Tổng quan về các loại hệ thống ML

Cho đến nay, chúng ta đã tìm hiểu rất nhiều nội dung: bạn đã biết học máy thực sự là gì, tại sao nó lại hữu ích, một số loại hệ thống ML phổ biến nhất là gì và quy trình làm việc điển hình của một dự án trông như thế nào. Bây giờ, hãy cùng xem những gì có thể xảy ra trong quá trình học và cản trở bạn đưa ra các dự đoán chính xác.

Những thách thức chính của học máy

Tóm lại, vì nhiệm vụ chính của bạn là chọn một mô hình và huấn luyện nó trên một số dữ liệu, hai điều có thể xảy ra sai sót là "mô hình kém" và "dữ liệu kém". Hãy bắt đầu với các ví dụ về dữ liệu kém.

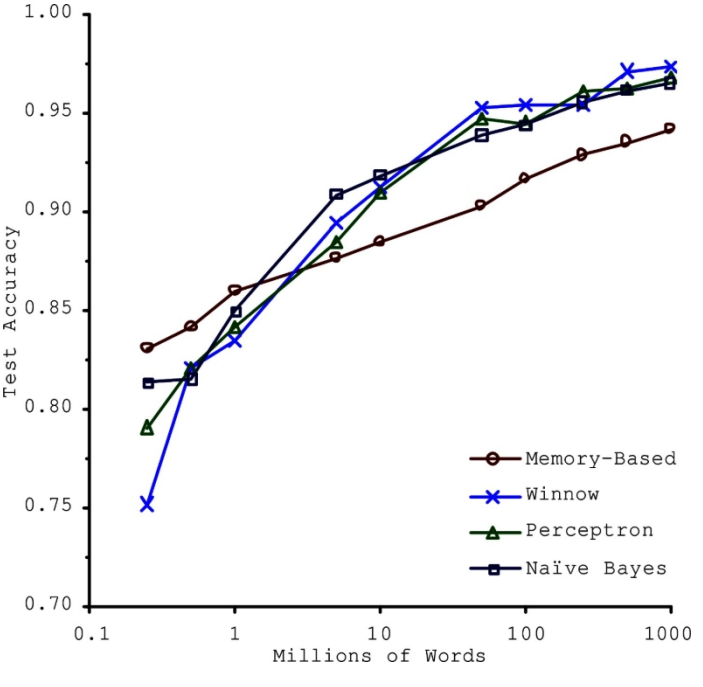
Lượng dữ liệu huấn luyện không đủ

Để một đứa trẻ mới biết đi học được "táo" là gì, tất cả những gì bạn cần làm là chỉ vào một quả táo và nói "táo" (có thể lặp lại quy trình này vài lần). Giờ đây, đứa trẻ đã có thể nhận diện táo dù chúng có màu sắc hay hình dạng nào đi chăng nữa. Thật tài tình.

Học máy vẫn chưa đạt đến mức đó; hầu hết các thuật toán học máy cần rất nhiều dữ liệu để hoạt động đúng. Ngay cả với những vấn đề rất đơn giản, bạn thường cần hàng nghìn ví dụ, và đối với những vấn đề phức tạp như nhận diện hình ảnh hoặc giọng nói, bạn có thể cần hàng triệu ví dụ (trừ khi bạn có thể tái sử dụng các phần của một mô hình hiện có, tức là học chuyển giao).

Hiệu quả phi thường của dữ liệu

Trong một bài báo nổi tiếng được công bố năm 2001, các nhà nghiên cứu của Microsoft là Michele Banko và Eric Brill đã chỉ ra rằng các thuật toán học máy rất khác nhau, bao gồm cả những thuật toán khá đơn giản, đều cho kết quả gần như giống hệt nhau khi giải quyết một vấn đề phức tạp về việc loại bỏ sự mơ hồ trong ngôn ngữ tự nhiên[6] một khi chúng được cung cấp đủ dữ liệu (như bạn có thể thấy trong Hình 1-21).



Hình 1-21. Tầm quan trọng của dữ liệu so với các thuật toán[7]

6 Ví dụ, biết nên viết "to", "two" hay "too" tùy theo ngữ cảnh.

7 Hình được sao chép với sự cho phép của Michele Banko và Eric Brill, "Scaling to Very Very Large Corpora for Natural Language Disambiguation", *Kỷ yếu Hội nghị thường niên lần thứ 39 của Hiệp hội Ngôn ngữ học Tính toán* (2001): 26–33.

Như các tác giả đã nêu, "các kết quả này cho thấy chúng ta có thể cần xem xét lại sự cân bằng
giữa việc dành thời gian và tiền bạc cho việc phát triển thuật toán so với việc dành chúng cho việc phát triển tập dữ liệu".

Ý tưởng rằng dữ liệu quan trọng hơn thuật toán đối với các vấn đề phức tạp đã được Peter Norvig và các cộng sự phổ biến rộng rãi hơn trong bài báo có tiêu đề "The Unreasonable Effectiveness of Data", được xuất bản năm 2009.[8] Tuy nhiên, cần lưu ý rằng các tập dữ liệu quy mô nhỏ và trung bình vẫn rất phổ biến, và việc thu thập thêm dữ liệu huấn luyện không phải lúc nào cũng dễ dàng hay rẻ tiền—vì vậy đừng vội từ bỏ các thuật toán.

Dữ liệu huấn luyện không đại diện

Để có khả năng tổng quát hóa tốt, điều quan trọng là dữ liệu huấn luyện của bạn phải đại diện cho các trường hợp mới mà bạn muốn tổng quát hóa. Điều này đúng cho dù bạn sử dụng học dựa trên thực thể hay học dựa trên mô hình.

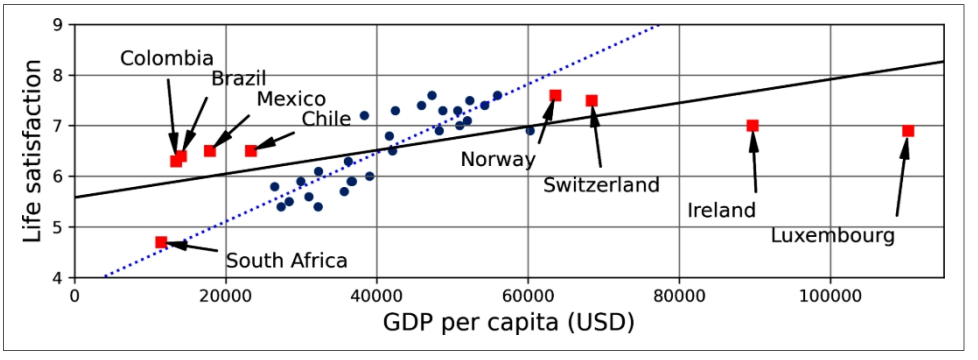
Ví dụ, tập hợp các quốc gia mà bạn đã sử dụng trước đó để huấn luyện mô hình tuyến tính không hoàn toàn đại diện; nó không chứa bất kỳ quốc gia nào có GDP bình quân đầu người thấp hơn 23.500 USD hoặc cao hơn 62.500 USD. Hình 1-22 cho thấy dữ liệu trông như thế nào khi bạn thêm các quốc gia như vậy.

Nếu bạn huấn luyện mô hình tuyến tính trên dữ liệu này, bạn sẽ nhận được đường liền nét, trong khi mô hình cũ được biểu diễn bằng đường chấm. Như bạn có thể thấy, việc thêm một vài quốc gia còn thiếu không chỉ làm thay đổi đáng kể mô hình, mà còn cho thấy rõ rằng một mô hình tuyến tính đơn giản như vậy có lẽ sẽ không bao giờ hoạt động tốt. Có vẻ như các quốc gia rất giàu không hạnh phúc hơn các quốc gia giàu vừa phải (thực tế, họ dường như còn hơi bất hạnh hơn một chút!), và ngược lại, một số quốc gia nghèo dường như hạnh phúc hơn nhiều quốc gia giàu.

Bằng cách sử dụng tập huấn luyện không đại diện, bạn đã huấn luyện một mô hình khó có thể đưa ra dự đoán chính xác, đặc biệt là đối với các quốc gia rất nghèo và rất giàu.

Việc sử dụng tập huấn luyện đại diện cho các trường hợp mà bạn muốn suy rộng là vô cùng quan trọng. Điều này thường khó hơn so với tưởng tượng: nếu mẫu quá nhỏ, bạn sẽ gặp phải *nhiễu lấy mẫu* (tức là dữ liệu không đại diện do ngẫu nhiên), nhưng ngay cả các mẫu rất lớn cũng có thể không đại diện nếu phương pháp lấy mẫu có sai sót. Điều này được gọi là *sai lệch lấy mẫu*.

8 Peter Norvig và cộng sự, "The Unreasonable Effectiveness of Data", *IEEE Intelligent Systems* 24, số 2 (2009): 8–12.



Hình 1-22. Một mẫu huấn luyện đại diện hơn

Ví dụ về sai lệch lấy mẫu

Có lẽ ví dụ nổi tiếng nhất về sai lệch lấy mẫu xảy ra trong cuộc bầu cử tổng thống Mỹ năm 1936, khi Landon đối đầu với Roosevelt: Tạp chí *Literary Digest* đã tiến hành một cuộc thăm dò quy mô rất lớn, gửi thư đến khoảng 10 triệu người. Họ nhận được 2,4 triệu câu trả lời và dự đoán với độ tin cậy cao rằng Landon sẽ giành được 57% số phiếu. Thay vào đó, Roosevelt đã thắng với 62% số phiếu. Sai sót nằm ở phương pháp lấy mẫu của tạp chí *Literary Digest*:

- Thứ nhất, để thu thập địa chỉ gửi phiếu khảo sát, *Literary Digest* đã sử dụng danh bạ điện thoại, danh sách người đăng ký tạp chí, danh sách thành viên câu lạc bộ và các nguồn tương tự. Tất cả các danh sách này đều có xu hướng nghiêng về những người giàu có, những người có khả năng cao hơn sẽ bỏ phiếu cho Đảng Cộng hòa (do đó ủng hộ Landon).
- Thứ hai, chỉ có dưới 25% số người được khảo sát trả lời. Điều này một lần nữa gây ra sai lệch lấy mẫu, do có thể loại trừ những người không quan tâm nhiều đến chính trị, những người không ưa Tạp *chí Literary Digest* và các nhóm quan trọng khác. Đây là một loại sai lệch lấy mẫu đặc biệt gọi là *sai lệch do không trả lời.*

Dưới đây là một ví dụ khác: giả sử bạn muốn xây dựng một hệ thống nhận diện video nhạc funk. Một cách để xây dựng tập huấn luyện là tìm kiếm "nhạc funk" trên YouTube và sử dụng các video thu được. Tuy nhiên, điều này giả định rằng công cụ tìm kiếm của YouTube trả về một tập hợp video đại diện cho tất cả các video nhạc funk trên YouTube. Trên thực tế, kết quả tìm kiếm có khả năng thiên về các nghệ sĩ nổi tiếng (và nếu bạn sống ở Brazil, bạn sẽ nhận được rất nhiều video "funk carioca", nghe không giống James Brown chút nào). Mặt khác, còn cách nào khác để bạn có được một tập dữ liệu huấn luyện lớn?

Dữ liệu chất lượng kém

Rõ ràng, nếu dữ liệu huấn luyện của bạn chứa đầy lỗi, giá trị ngoại lệ và nhiễu (ví dụ: do các phép đo chất lượng kém), hệ thống sẽ khó phát hiện các mẫu cơ bản hơn, do đó hệ thống của bạn sẽ ít có khả năng hoạt động tốt. Thường thì việc dành thời gian để làm sạch dữ liệu huấn luyện là rất đáng giá. Thực tế là, hầu hết các nhà khoa học dữ liệu đều dành một phần đáng kể thời gian của họ chỉ để làm việc đó. Dưới đây là một vài ví dụ về những trường hợp bạn nên làm sạch dữ liệu huấn luyện:

- Nếu một số trường hợp rõ ràng là giá trị ngoại lệ, bạn có thể loại bỏ chúng hoặc cố gắng sửa lỗi thủ công.
- Nếu một số trường hợp thiếu một vài đặc trưng (ví dụ: 5% khách hàng của bạn không nêu rõ độ tuổi), bạn phải quyết định xem có muốn bỏ qua hoàn toàn đặc trưng này, bỏ qua các trường hợp đó, điền vào các giá trị còn thiếu (ví dụ: bằng độ tuổi trung vị), hay huấn luyện một mô hình có đặc trưng này và một mô hình không có đặc trưng đó.

Các thuộc tính không liên quan

Như câu nói: "rác vào, rác ra". Hệ thống của bạn sẽ chỉ có khả năng học nếu dữ liệu huấn luyện chứa đủ các đặc tính có liên quan và không có quá nhiều đặc tính không liên quan. Một phần quan trọng dẫn đến thành công của dự án học máy là đưa ra một tập hợp các đặc tính tốt để huấn luyện. Quá trình này, được gọi là *kỹ thuật đặc tính*, bao gồm các bước sau:

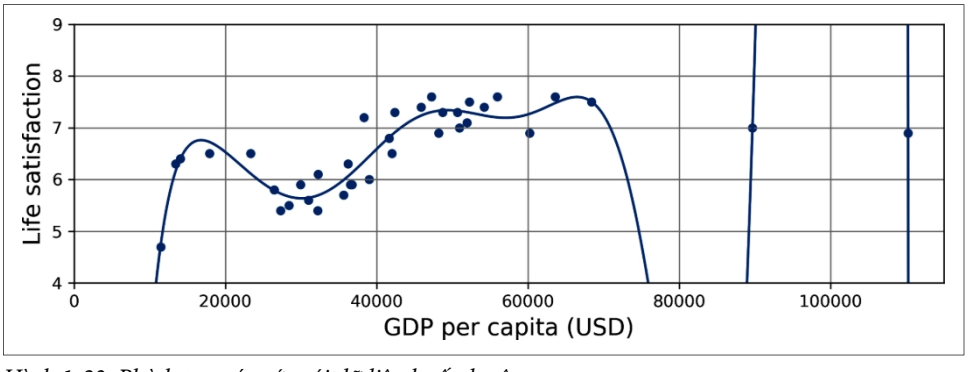
- *Lựa chọn đặc trưng* (chọn ra các đặc trưng hữu ích nhất để huấn luyện trong số các đặc trưng hiện có)
- *Trích xuất đặc trưng* (kết hợp các đặc trưng hiện có để tạo ra một đặc trưng hữu ích hơn — như chúng ta đã thấy trước đó, các thuật toán giảm chiều có thể giúp ích)
- Tạo các đặc trưng mới bằng cách thu thập dữ liệu mới

Sau khi đã xem xét nhiều ví dụ về dữ liệu kém chất lượng, hãy cùng xem xét một vài ví dụ về các thuật toán kém chất lượng.

Quá khớp với dữ liệu huấn luyện

Giả sử bạn đang du lịch ở một quốc gia nước ngoài và tài xế taxi lừa gạt bạn. Bạn có thể bị cám dỗ để kết luận rằng *tất cả* tài xế taxi ở quốc gia đó đều là kẻ trộm. Việc suy diễn quá mức là điều con người chúng ta thường mắc phải, và đáng tiếc là máy móc cũng có thể rơi vào cùng một bẫy nếu chúng ta không cẩn thận. Trong học máy, hiện tượng này được gọi *là quá khớp*: nghĩa là mô hình hoạt động tốt trên dữ liệu huấn luyện, nhưng không thể suy diễn tốt trên dữ liệu mới.

Hình 1-23 minh họa một ví dụ về mô hình độ hài lòng với cuộc sống dạng đa thức bậc cao bị quá khớp dữ liệu huấn luyện nghiêm trọng. Mặc dù mô hình này hoạt động tốt hơn nhiều so với mô hình tuyến tính đơn giản trên dữ liệu huấn luyện, liệu bạn có thực sự tin tưởng vào các dự đoán của nó không?



Hình 1-23. Phù hợp quá mức với dữ liệu huấn luyện

Các mô hình phức tạp như mạng nơ-ron sâu có thể phát hiện các mẫu tinh tế trong dữ liệu, nhưng nếu tập huấn luyện có nhiễu hoặc quá nhỏ (gây ra nhiễu lấy mẫu), thì mô hình có khả năng sẽ phát hiện các mẫu ngay trong chính nhiễu đó (như trong ví dụ về tài xế taxi). Rõ ràng, những mẫu này sẽ không thể tổng quát hóa sang các trường hợp mới. Ví dụ, giả sử bạn cung cấp cho mô hình đánh giá mức độ hài lòng với cuộc sống nhiều thuộc tính hơn, bao gồm cả những thuộc tính không mang thông tin như tên quốc gia. Trong trường hợp đó, một mô hình phức tạp có thể phát hiện các mẫu như việc tất cả các quốc gia trong tập dữ liệu huấn luyện có chữ *"w"* trong tên đều có mức độ hài lòng với cuộc sống lớn hơn 7: New Zealand (7,3), Na Uy (7,6), Thụy Điển (7,3) và Thụy Sĩ (7,5). Bạn có tự tin rằng quy tắc "chữ w – mức độ hài lòng" này có thể áp dụng cho Rwanda hay Zimbabwe không? Rõ ràng mẫu này xuất hiện trong dữ liệu huấn luyện chỉ là do ngẫu nhiên, nhưng mô hình không có cách nào để phân biệt liệu một mẫu có thật hay chỉ đơn thuần là kết quả của nhiễu trong dữ liệu.

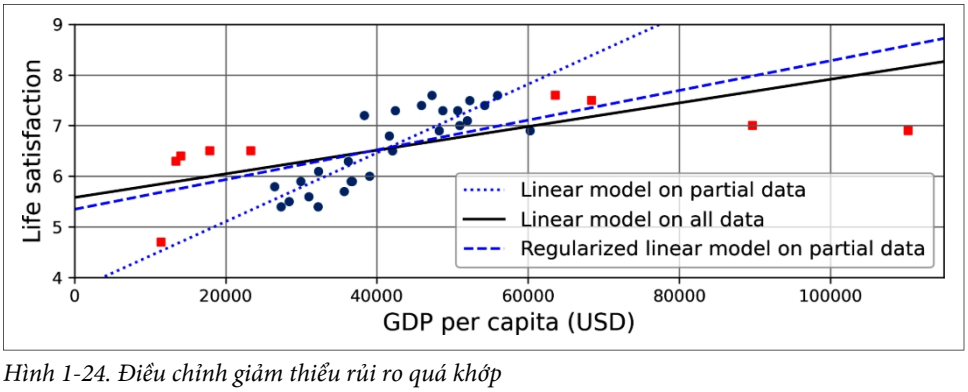


Hiện tượng quá khớp xảy ra khi mô hình quá phức tạp so với khối lượng và độ nhiễu của dữ liệu huấn luyện, do đó mô hình bắt đầu học các mẫu ngẫu nhiên trong dữ liệu huấn luyện. Dưới đây là các giải pháp khả thi:

- Đơn giản hóa mô hình bằng cách chọn mô hình có ít tham số hơn (ví dụ: mô hình tuyến tính thay vì mô hình đa thức bậc cao), bằng cách giảm số lượng thuộc tính trong dữ liệu huấn luyện hoặc bằng cách hạn chế mô hình.
- Thu thập thêm dữ liệu huấn luyện.
- Giảm nhiễu trong dữ liệu huấn luyện (ví dụ: sửa lỗi dữ liệu và loại bỏ các giá trị ngoại lệ).

Việc đặt ràng buộc lên mô hình để làm cho nó đơn giản hơn và giảm nguy cơ quá khớp được gọi *là chính quy hóa*. Ví dụ, mô hình tuyến tính mà chúng ta đã định nghĩa trước đó có hai tham số, $\theta_0$và θ_1. Điều này cung cấp cho thuật toán học hai *bậc tự do* để điều chỉnh mô hình phù hợp với dữ liệu huấn luyện: nó có thể điều chỉnh cả độ cao (θ_0) và độ dốc (θ_1) của đường thẳng. Nếu chúng ta ép θ_1= 0, thuật toán sẽ chỉ còn một bậc tự do và sẽ gặp nhiều khó khăn hơn trong việc vừa khít dữ liệu một cách chính xác: tất cả những gì nó có thể làm là di chuyển đường thẳng lên hoặc xuống để đến gần nhất có thể với các mẫu huấn luyện, do đó nó sẽ nằm quanh giá trị trung bình. Thật sự là một mô hình rất đơn giản! Nếu chúng ta cho phép thuật toán điều chỉnh $\theta_1$nhưng buộc nó phải giữ giá trị này ở mức nhỏ, thì thuật toán học sẽ thực tế có độ tự do nằm giữa một và hai. Nó sẽ tạo ra một mô hình đơn giản hơn so với mô hình có hai độ tự do, nhưng phức tạp hơn so với mô hình chỉ có một độ tự do. Bạn cần tìm sự cân bằng phù hợp giữa việc vừa khít dữ liệu huấn luyện một cách hoàn hảo và giữ cho mô hình đủ đơn giản để đảm bảo khả năng tổng quát hóa tốt.

Hình 1-24 trình bày ba mô hình. Đường chấm chấm đại diện cho mô hình ban đầu được huấn luyện trên các quốc gia được biểu diễn bằng hình tròn (không bao gồm các quốc gia được biểu diễn bằng hình vuông), đường liền nét là mô hình thứ hai được huấn luyện với tất cả các quốc gia (hình tròn và hình vuông), còn đường gạch ngang là mô hình được huấn luyện với cùng dữ liệu như mô hình đầu tiên nhưng có thêm ràng buộc chính quy hóa. Bạn có thể thấy rằng điều chỉnh đã buộc mô hình phải có độ dốc nhỏ hơn: mô hình này không phù hợp với dữ liệu huấn luyện (hình tròn) tốt như mô hình đầu tiên, nhưng thực tế nó lại tổng quát hóa tốt hơn đối với các ví dụ mới mà nó chưa từng gặp trong quá trình huấn luyện (hình vuông).



Hình 1-24. Điều chỉnh giảm thiểu rủi ro quá khớp

Mức độ chính quy hóa áp dụng trong quá trình học có thể được điều chỉnh bằng một *siêu tham số*. Siêu tham số là tham số của thuật toán học (không phải của mô hình). Do đó, nó không bị ảnh hưởng bởi chính thuật toán học; nó phải được thiết lập trước khi huấn luyện và giữ nguyên trong suốt quá trình huấn luyện. Nếu bạn đặt siêu tham số chính quy hóa thành một giá trị rất lớn, bạn sẽ nhận được một mô hình gần như bằng phẳng (độ dốc gần bằng không); thuật toán học gần như chắc chắn sẽ không bị quá khớp với dữ liệu huấn luyện, nhưng khả năng tìm ra một giải pháp tốt sẽ thấp hơn. Điều chỉnh các siêu tham số là một

trong việc xây dựng hệ thống học máy (bạn sẽ thấy một ví dụ chi tiết trong chương tiếp theo).

Dữ liệu huấn luyện bị thiếu phù hợp

Như bạn có thể đoán, *hiện tượng thiếu phù hợp* là ngược lại với hiện tượng quá phù hợp: nó xảy ra khi mô hình của bạn quá đơn giản để học được cấu trúc cơ bản của dữ liệu. Ví dụ, một mô hình tuyến tính về mức độ hài lòng với cuộc sống dễ bị thiếu phù hợp; thực tế phức tạp hơn mô hình, do đó các dự đoán của nó chắc chắn sẽ không chính xác, ngay cả trên các ví dụ huấn luyện.

Dưới đây là các phương án chính để khắc phục vấn đề này:

- Chọn một mô hình mạnh mẽ hơn, với nhiều tham số hơn.
- Cung cấp các đặc trưng tốt hơn cho thuật toán học (kỹ thuật đặc trưng).
- Giảm bớt các ràng buộc đối với mô hình (ví dụ bằng cách giảm tham số siêu quy chuẩn).

Các vấn đề khi triển khai

Ngay cả khi bạn có một tập dữ liệu lớn và sạch, và bạn đã huấn luyện được một mô hình hoàn hảo không bị underfitting hay overfitting, bạn vẫn có thể gặp phải các vấn đề trong quá trình triển khai: ví dụ, mô hình có thể quá phức tạp để duy trì, quá lớn để chứa vừa trong bộ nhớ, quá chậm, không mở rộng quy mô đúng cách, có lỗ hổng bảo mật, hoặc trở nên lỗi thời nếu không được cập nhật thường xuyên, v.v.

Nói tóm lại, một dự án ML không chỉ đơn thuần là dữ liệu và mô hình. Tuy nhiên, các kỹ năng cần thiết để xử lý những vấn đề vận hành này khá khác biệt so với những kỹ năng cần thiết cho việc mô hình hóa dữ liệu, đó là lý do tại sao các công ty thường có một đội ngũ MLOps (vận hành ML) chuyên trách để xử lý vấn đề này.

Nhìn lại

Đến đây, bạn đã biết rất nhiều về học máy. Tuy nhiên, chúng ta đã đi qua rất nhiều khái niệm nên có thể bạn đang cảm thấy hơi bối rối, vì vậy hãy lùi lại một bước và nhìn vào bức tranh tổng thể:

- Học máy là việc giúp máy móc thực hiện tốt hơn một số tác vụ bằng cách học từ dữ liệu, thay vì phải viết mã các quy tắc một cách rõ ràng.
- Có nhiều loại hệ thống ML khác nhau: có giám sát hoặc không, theo lô hoặc trực tuyến, dựa trên trường hợp hoặc dựa trên mô hình.
- Trong một dự án học máy, bạn thu thập dữ liệu vào tập huấn luyện, sau đó đưa tập huấn luyện này vào thuật toán học. Nếu thuật toán dựa trên mô hình, nó sẽ điều chỉnh một số tham số để phù hợp mô hình với tập huấn luyện (tức là để đưa ra dự đoán chính xác trên chính tập huấn luyện

chính nó), và sau đó hy vọng nó cũng có thể đưa ra dự đoán chính xác cho các trường hợp mới. Nếu thuật toán dựa trên thực thể, nó chỉ ghi nhớ các ví dụ và tổng quát hóa sang các thực thể mới bằng cách sử dụng thước đo độ tương đồng để so sánh chúng với các thực thể đã học.

- Hệ thống sẽ không hoạt động tốt nếu tập huấn luyện quá nhỏ, hoặc nếu dữ liệu không đại diện, bị nhiễu, hoặc bị ô nhiễm bởi các đặc trưng không liên quan (rác vào, rác ra). Mô hình của bạn không nên quá đơn giản (trong trường hợp đó nó sẽ bị thiếu phù hợp) cũng không nên quá phức tạp (trong trường hợp đó nó sẽ bị quá phù hợp). Cuối cùng, bạn phải cân nhắc kỹ lưỡng các ràng buộc khi triển khai.

Còn một chủ đề quan trọng cuối cùng cần đề cập: một khi bạn đã huấn luyện xong mô hình, bạn không nên chỉ "hy vọng" rằng nó sẽ khái quát hóa được các trường hợp mới. Bạn cần đánh giá mô hình và tinh chỉnh nếu cần thiết. Hãy cùng xem cách thực hiện điều đó.

Kiểm thử và Xác thực

Cách duy nhất để biết mô hình sẽ tổng quát hóa tốt như thế nào đối với các trường hợp mới là thực sự thử nghiệm nó trên các trường hợp mới. Một cách để làm điều này là đưa mô hình vào sản xuất và theo dõi hiệu suất của nó. Cách này hoạt động tốt, nhưng nếu mô hình của bạn cực kỳ tệ, người dùng sẽ phàn nàn — đây không phải là ý tưởng hay nhất.

Một lựa chọn tốt hơn là chia dữ liệu thành hai tập: *tập huấn luyện* và *tập kiểm tra*. Như tên gọi đã gợi ý, bạn huấn luyện mô hình bằng tập huấn luyện và kiểm tra mô hình bằng tập kiểm tra. Tỷ lệ lỗi trên các trường hợp mới được gọi là *lỗi tổng quát hóa* (hoặc *lỗi ngoài mẫu*), và bằng cách đánh giá mô hình trên tập kiểm tra, bạn sẽ có được ước tính về lỗi này. Giá trị này cho biết mô hình của bạn sẽ hoạt động tốt như thế nào trên các trường hợp mà nó chưa từng gặp trước đây.

Nếu sai số huấn luyện thấp (tức là mô hình của bạn mắc ít lỗi trên tập huấn luyện) nhưng sai số tổng quát hóa cao, điều đó có nghĩa là mô hình của bạn đang bị quá khớp với dữ liệu huấn luyện.



Thông thường, người ta sử dụng 80% dữ liệu để huấn luyện và *giữ lại* 20% để kiểm tra. Tuy nhiên, điều này phụ thuộc vào kích thước của tập dữ liệu: nếu tập dữ liệu chứa 10 triệu trường hợp, thì việc giữ lại 1% có nghĩa là tập kiểm tra của bạn sẽ chứa 100.000 trường hợp, có lẽ là quá đủ để có được ước tính tốt về sai số tổng quát.

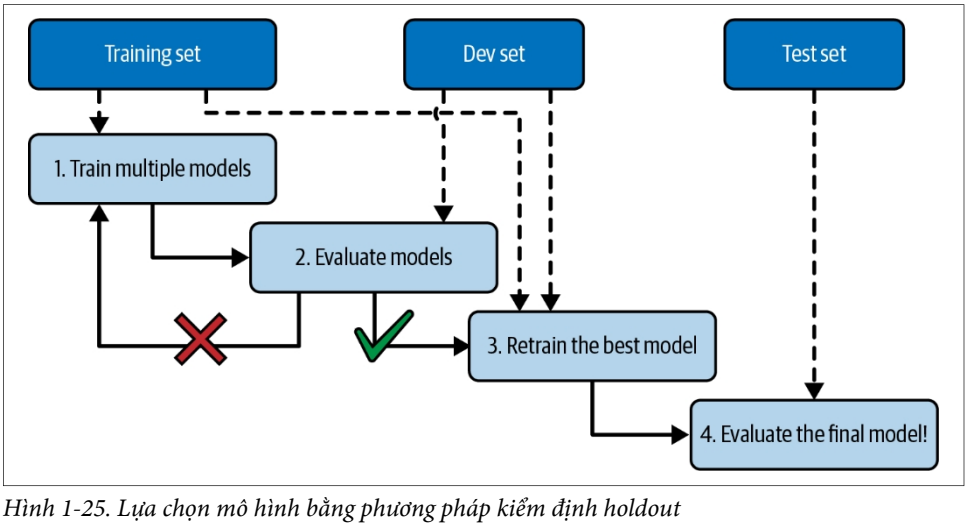
Điều chỉnh siêu tham số và lựa chọn mô hình

Đánh giá mô hình khá đơn giản: chỉ cần sử dụng tập kiểm tra. Nhưng giả sử bạn đang phân vân giữa hai loại mô hình (ví dụ: mô hình tuyến tính và mô hình đa thức): làm thế nào để quyết định chọn mô hình nào? Một lựa chọn là huấn luyện cả hai và so sánh khả năng tổng quát hóa của chúng thông qua tập kiểm tra.

Giờ giả sử mô hình tuyến tính tổng quát hóa tốt hơn, nhưng bạn muốn áp dụng một số phương pháp thường quy hóa để tránh hiện tượng quá khớp. Câu hỏi đặt ra là, làm thế nào để chọn giá trị của siêu tham số thường quy hóa? Một lựa chọn là huấn luyện 100 mô hình khác nhau bằng cách sử dụng 100 giá trị khác nhau cho siêu tham số này. Giả sử bạn tìm được giá trị siêu tham số tối ưu tạo ra mô hình có sai số tổng quát hóa thấp nhất — ví dụ, chỉ 5% sai số. Bạn triển khai mô hình này vào sản xuất, nhưng đáng tiếc là nó không hoạt động tốt như mong đợi và tạo ra sai số 15%. Điều gì vừa xảy ra?

Vấn đề là bạn đã đo lường sai số tổng quát hóa nhiều lần trên tập kiểm tra, và bạn đã điều chỉnh mô hình cùng các siêu tham số để tạo ra mô hình tốt nhất *cho tập dữ liệu cụ thể đó*. Điều này có nghĩa là mô hình khó có thể hoạt động tốt trên dữ liệu mới.

Một giải pháp phổ biến cho vấn đề này được gọi là *kiểm định holdout* (Hình 1-25): bạn chỉ cần tách một phần của tập huấn luyện ra để đánh giá nhiều mô hình ứng viên và chọn mô hình tốt nhất. Tập dữ liệu mới được tách ra này được gọi là *tập kiểm định* (hoặc *tập phát triển*, hay *dev set*). Cụ thể hơn, bạn huấn luyện nhiều mô hình với các siêu tham số khác nhau trên tập huấn luyện đã được thu hẹp (tức là tập huấn luyện đầy đủ trừ đi tập xác thực), và chọn mô hình hoạt động tốt nhất trên tập xác thực. Sau quá trình xác thực bằng cách giữ lại này, bạn huấn luyện mô hình tốt nhất trên tập huấn luyện đầy đủ (bao gồm cả tập xác thực), và điều này cho bạn mô hình cuối cùng. Cuối cùng, bạn đánh giá mô hình cuối cùng này trên tập kiểm tra để ước tính sai số tổng quát hóa.



Hình 1-25. Lựa chọn mô hình bằng phương pháp kiểm định holdout

Giải pháp này thường hoạt động khá hiệu quả. Tuy nhiên, nếu tập kiểm định quá nhỏ, việc đánh giá mô hình sẽ thiếu chính xác: bạn có thể vô tình chọn phải một mô hình không tối ưu. Ngược lại, nếu tập kiểm định quá lớn, tập huấn luyện còn lại sẽ nhỏ hơn nhiều so với tập huấn luyện đầy đủ. Tại sao điều này lại không tốt? Bởi vì mô hình cuối cùng sẽ được huấn luyện trên tập huấn luyện đầy đủ, nên việc so sánh

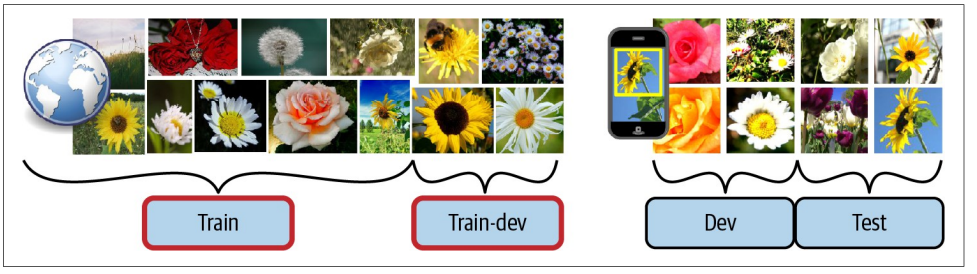
các mô hình ứng viên được huấn luyện trên tập huấn luyện nhỏ hơn nhiều. Điều này giống như việc chọn vận động viên chạy nước rút nhanh nhất để tham gia một cuộc đua marathon. Một cách để giải quyết vấn đề này là thực hiện *kiểm định chéo* lặp lại, sử dụng nhiều tập kiểm định nhỏ. Mỗi mô hình được đánh giá một lần trên mỗi tập kiểm định sau khi được huấn luyện trên phần dữ liệu còn lại. Bằng cách tính trung bình tất cả các kết quả đánh giá của một mô hình, bạn sẽ có được thước đo chính xác hơn nhiều về hiệu suất của nó. Tuy nhiên, có một nhược điểm: thời gian huấn luyện sẽ tăng lên gấp bội theo số lượng tập dữ liệu kiểm chứng.

Sự không phù hợp của dữ liệu

Trong một số trường hợp, việc thu thập một lượng lớn dữ liệu để huấn luyện là khá dễ dàng, nhưng dữ liệu này có thể sẽ không hoàn toàn đại diện cho dữ liệu sẽ được sử dụng trong môi trường sản xuất. Ví dụ, giả sử bạn muốn tạo một ứng dụng di động để chụp ảnh hoa và tự động xác định loài của chúng. Bạn có thể dễ dàng tải xuống hàng triệu bức ảnh hoa trên mạng, nhưng chúng sẽ không hoàn toàn đại diện cho những bức ảnh thực tế được chụp bằng ứng dụng trên thiết bị di động. Có thể bạn chỉ có 1.000 bức ảnh đại diện (tức là thực sự được chụp bằng ứng dụng).

Trong trường hợp này, quy tắc quan trọng nhất cần nhớ là cả tập dữ liệu kiểm định và tập dữ liệu thử nghiệm đều phải đại diện nhất có thể cho dữ liệu bạn dự kiến sử dụng trong môi trường sản xuất, do đó chúng nên chỉ bao gồm các bức ảnh đại diện: bạn có thể trộn chúng lại và chia một nửa vào tập dữ liệu kiểm định và một nửa vào tập dữ liệu thử nghiệm (đảm bảo không có ảnh trùng lặp hoặc gần trùng lặp xuất hiện trong cả hai tập). Sau khi huấn luyện mô hình trên các hình ảnh từ web, nếu bạn nhận thấy hiệu suất của mô hình trên tập xác thực không như mong đợi, bạn sẽ không biết liệu điều này là do mô hình của bạn đã bị quá khớp với tập huấn luyện, hay chỉ đơn thuần là do sự không phù hợp giữa các hình ảnh từ web và các hình ảnh từ ứng dụng di động.

Một giải pháp là tách một phần các hình ảnh huấn luyện (từ web) ra thành một tập dữ liệu khác mà Andrew Ng gọi là *tập train-dev* (Hình 1-26). Sau khi mô hình được huấn luyện (trên tập huấn luyện, *không phải* trên tập train-dev), bạn có thể đánh giá mô hình trên tập train-dev. Nếu mô hình hoạt động kém, thì có nghĩa là nó đã bị quá khớp với tập huấn luyện, do đó bạn nên cố gắng đơn giản hóa hoặc chuẩn hóa mô hình, thu thập thêm dữ liệu huấn luyện và làm sạch dữ liệu huấn luyện. Nhưng nếu mô hình hoạt động tốt trên tập train-dev, thì bạn có thể đánh giá mô hình trên tập dev. Nếu mô hình hoạt động kém, thì vấn đề chắc chắn xuất phát từ sự không khớp giữa dữ liệu. Bạn có thể giải quyết vấn đề này bằng cách tiền xử lý các hình ảnh trên web để chúng trông giống hơn với các bức ảnh sẽ được chụp bởi ứng dụng di động, sau đó huấn luyện lại mô hình. Khi đã có mô hình hoạt động tốt trên cả tập huấn luyện-phát triển và tập phát triển, bạn có thể đánh giá nó lần cuối trên tập kiểm tra để biết mô hình có khả năng hoạt động tốt như thế nào trong môi trường sản xuất.



Hình 1-26. Khi dữ liệu thực tế khan hiếm (bên phải), bạn có thể sử dụng dữ liệu tương tự dồi dào (bên trái) để huấn luyện và giữ lại một phần trong tập train-dev để đánh giá hiện tượng quá khớp; dữ liệu thực tế sau đó được dùng để đánh giá sự không khớp dữ liệu (tập dev) và đánh giá hiệu suất của mô hình cuối cùng (tập test)

Định lý "Không có bữa trưa miễn phí"

Mô hình là một biểu diễn đơn giản hóa của dữ liệu. Các đơn giản hóa này nhằm loại bỏ những chi tiết thừa thãi ít có khả năng tổng quát hóa sang các bản ghi mới. Khi bạn chọn một loại mô hình cụ thể, bạn đang ngầm *giả định* về dữ liệu. Ví dụ, nếu bạn chọn mô hình tuyến tính, bạn đang ngầm giả định rằng dữ liệu về cơ bản là tuyến tính và khoảng cách giữa các bản ghi và đường thẳng chỉ là nhiễu, có thể an toàn bỏ qua.

Trong một bài báo nổi tiếng năm 1996,[9] David Wolpert đã chứng minh rằng nếu bạn không đưa ra bất kỳ giả định nào về dữ liệu, thì không có lý do gì để ưu tiên một mô hình này hơn mô hình khác. Điều này được gọi là định lý "*Không có bữa trưa miễn phí*" (NFL). Đối với một số tập dữ liệu, mô hình tốt nhất là mô hình tuyến tính, trong khi đối với các tập dữ liệu khác, đó là mạng nơ-ron. Không có mô hình nào được đảm bảo *trước* là sẽ hoạt động tốt hơn (do đó tên của định lý). Cách duy nhất để biết chắc chắn mô hình nào là tốt nhất là đánh giá tất cả các mô hình. Vì điều này là không thể, trên thực tế, bạn sẽ đưa ra một số giả định hợp lý về dữ liệu và chỉ đánh giá một vài mô hình hợp lý. Ví dụ, đối với các tác vụ đơn giản, bạn có thể đánh giá các mô hình tuyến tính với các mức độ chính quy hóa khác nhau, còn đối với một vấn đề phức tạp, bạn có thể đánh giá các mạng nơ-ron khác nhau.

9 David Wolpert, "The Lack of A Priori Distinctions Between Learning Algorithms", *Neural Computation* 8, số 7 (1996): 1341–1390.

Bài tập

Trong chương này, chúng ta đã tìm hiểu một số khái niệm quan trọng nhất trong học máy. Trong các chương tiếp theo, chúng ta sẽ đi sâu hơn và viết thêm mã, nhưng trước khi làm điều đó, hãy đảm bảo rằng bạn có thể trả lời các câu hỏi sau:

1. Bạn sẽ định nghĩa học máy như thế nào?
2. Bạn có thể nêu tên bốn loại ứng dụng mà học máy phát huy hiệu quả nhất không?
3. Bộ dữ liệu huấn luyện có nhãn là gì?
4. Hai tác vụ có giám sát phổ biến nhất là gì?
5. Bạn có thể nêu tên bốn tác vụ không có giám sát phổ biến không?
6. Bạn sẽ sử dụng loại thuật toán nào để cho phép một robot di chuyển trên các địa hình chưa biết khác nhau?
7. Bạn sẽ sử dụng loại thuật toán nào để phân chia khách hàng thành nhiều nhóm?
8. Bạn sẽ định nghĩa vấn đề phát hiện thư rác là một vấn đề học có giám sát hay một vấn đề học không giám sát?
9. Hệ thống học trực tuyến là gì?
10. Học ngoài bộ nhớ là gì?
11. Loại thuật toán nào dựa vào thước đo độ tương đồng để đưa ra dự đoán?
12. Sự khác biệt giữa tham số mô hình và siêu tham số mô hình là gì?
13. Các thuật toán dựa trên mô hình tìm kiếm điều gì? Chiến lược phổ biến nhất mà chúng sử dụng để thành công là gì? Chúng đưa ra dự đoán như thế nào?
14. Bạn có thể nêu tên bốn thách thức chính trong học máy không?
15. Nếu mô hình của bạn hoạt động rất tốt trên dữ liệu huấn luyện nhưng lại không thể tổng quát hóa tốt sang các trường hợp mới, thì điều gì đang xảy ra? Bạn có thể nêu tên ba giải pháp khả thi không?
16. Bộ dữ liệu kiểm tra là gì, và tại sao bạn lại muốn sử dụng nó?
17. Mục đích của tập xác thực là gì?
18. Bộ dữ liệu train-dev là gì, khi nào bạn cần nó và cách sử dụng nó như thế nào?
19. Điều gì có thể xảy ra nếu bạn điều chỉnh các siêu tham số bằng cách sử dụng tập dữ liệu kiểm tra?

Các giải pháp cho các bài tập này có sẵn ở cuối notebook của chương này, tại *https://homl.info/colab-p*.